

In [1]: `import pandas as pd`

In [2]: `mve=pd.read_csv(r"C:\Users\Tharuni\Desktop\NIT\july month\24th_Ollama\Movie-Rati`

In [3]: `mve`

Out[3]:

|     | Film                 | Genre     | Rotten Tomatoes Ratings % | Audience Ratings % | Budget (million \$) | Year of release |
|-----|----------------------|-----------|---------------------------|--------------------|---------------------|-----------------|
| 0   | (500) Days of Summer | Comedy    | 87                        | 81                 | 8                   | 2009            |
| 1   | 10,000 B.C.          | Adventure | 9                         | 44                 | 105                 | 2008            |
| 2   | 12 Rounds            | Action    | 30                        | 52                 | 20                  | 2009            |
| 3   | 127 Hours            | Adventure | 93                        | 84                 | 18                  | 2010            |
| 4   | 17 Again             | Comedy    | 55                        | 70                 | 20                  | 2009            |
| ... | ...                  | ...       | ...                       | ...                | ...                 | ...             |
| 554 | Your Highness        | Comedy    | 26                        | 36                 | 50                  | 2011            |
| 555 | Youth in Revolt      | Comedy    | 68                        | 52                 | 18                  | 2009            |
| 556 | Zodiac               | Thriller  | 89                        | 73                 | 65                  | 2007            |
| 557 | Zombieland           | Action    | 90                        | 87                 | 24                  | 2009            |
| 558 | Zookeeper            | Comedy    | 14                        | 42                 | 80                  | 2011            |

559 rows × 6 columns

In [4]: `len(mve)`

Out[4]: 559

In [5]: `pd.__version__`

Out[5]: '2.2.2'

In [6]: `mve.columns`

Out[6]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %', 'Budget (million \$)', 'Year of release'], dtype='object')

In [7]: `mve.head()`

Out[7]:

|   | Film                 | Genre     | Rotten Tomatoes Ratings % | Audience Ratings % | Budget (million \$) | Year of release |
|---|----------------------|-----------|---------------------------|--------------------|---------------------|-----------------|
| 0 | (500) Days of Summer | Comedy    | 87                        | 81                 | 8                   | 2009            |
| 1 | 10,000 B.C.          | Adventure | 9                         | 44                 | 105                 | 2008            |
| 2 | 12 Rounds            | Action    | 30                        | 52                 | 20                  | 2009            |
| 3 | 127 Hours            | Adventure | 93                        | 84                 | 18                  | 2010            |
| 4 | 17 Again             | Comedy    | 55                        | 70                 | 20                  | 2009            |

In [8]: `mve.shape`

Out[8]: (559, 6)

In [9]: `mve.tail()`

Out[9]:

|     | Film            | Genre    | Rotten Tomatoes Ratings % | Audience Ratings % | Budget (million \$) | Year of release |
|-----|-----------------|----------|---------------------------|--------------------|---------------------|-----------------|
| 554 | Your Highness   | Comedy   | 26                        | 36                 | 50                  | 2011            |
| 555 | Youth in Revolt | Comedy   | 68                        | 52                 | 18                  | 2009            |
| 556 | Zodiac          | Thriller | 89                        | 73                 | 65                  | 2007            |
| 557 | Zombieland      | Action   | 90                        | 87                 | 24                  | 2009            |
| 558 | Zookeeper       | Comedy   | 14                        | 42                 | 80                  | 2011            |

In [10]: `mve.columns`

Out[10]: Index(['Film', 'Genre', 'Rotten Tomatoes Ratings %', 'Audience Ratings %', 'Budget (million \$)', 'Year of release'], dtype='object')

In [11]: `mve.columns = ['Film', 'Genre', 'CriticRating', 'AudienceRatings', 'BudgetMillions', 'Year']`

In [12]: `mve.head(1)`

Out[12]:

|   | Film                 | Genre  | CriticRating | AudienceRatings | BudgetMillions | Year |
|---|----------------------|--------|--------------|-----------------|----------------|------|
| 0 | (500) Days of Summer | Comedy | 87           | 81              | 8              | 2009 |

In [13]: `mve.shape`

Out[13]: (559, 6)

In [14]: `mve.describe()`

Out[14]:

|              | CriticRating | AudienceRatings | BudgetMillions | Year        |
|--------------|--------------|-----------------|----------------|-------------|
| <b>count</b> | 559.000000   | 559.000000      | 559.000000     | 559.000000  |
| <b>mean</b>  | 47.309481    | 58.744186       | 50.236136      | 2009.152057 |
| <b>std</b>   | 26.413091    | 16.826887       | 48.731817      | 1.362632    |
| <b>min</b>   | 0.000000     | 0.000000        | 0.000000       | 2007.000000 |
| <b>25%</b>   | 25.000000    | 47.000000       | 20.000000      | 2008.000000 |
| <b>50%</b>   | 46.000000    | 58.000000       | 35.000000      | 2009.000000 |
| <b>75%</b>   | 70.000000    | 72.000000       | 65.000000      | 2010.000000 |
| <b>max</b>   | 97.000000    | 96.000000       | 300.000000     | 2011.000000 |

In [15]: `mve.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Film             559 non-null   object
1   Genre            559 non-null   object
2   CriticRating     559 non-null   int64
3   AudienceRatings  559 non-null   int64
4   BudgetMillions   559 non-null   int64
5   Year             559 non-null   int64
dtypes: int64(4), object(2)
memory usage: 26.3+ KB
```

In [16]: `mve.Film = mve.Film.astype('category')`

In [17]: `mve.Film`

Out[17]:

```
0      (500) Days of Summer
1      10,000 B.C.
2      12 Rounds
3      127 Hours
4      17 Again
...
554     Your Highness
555     Youth in Revolt
556     Zodiac
557     Zombieland
558     Zookeeper
Name: Film, Length: 559, dtype: category
Categories (559, object): ['(500) Days of Summer ', '10,000 B.C.', '12 Rounds ', '127 Hours', ..., 'Youth in Revolt', 'Zodiac', 'Zombieland ', 'Zookeeper']
```

In [18]: `mve.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Film                  559 non-null    category
1   Genre                 559 non-null    object
2   CriticRating          559 non-null    int64
3   AudienceRatings      559 non-null    int64
4   BudgetMillions        559 non-null    int64
5   Year                  559 non-null    int64
dtypes: category(1), int64(4), object(1)
memory usage: 43.6+ KB
```

```
In [19]: mve.Genre = mve.Genre.astype('category')
mve.Year = mve.Year.astype('category')
```

```
In [20]: mve.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 559 entries, 0 to 558
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Film                  559 non-null    category
1   Genre                 559 non-null    category
2   CriticRating          559 non-null    int64
3   AudienceRatings      559 non-null    int64
4   BudgetMillions        559 non-null    int64
5   Year                  559 non-null    category
dtypes: category(3), int64(3)
memory usage: 36.5 KB
```

```
In [21]: mve.describe()
```

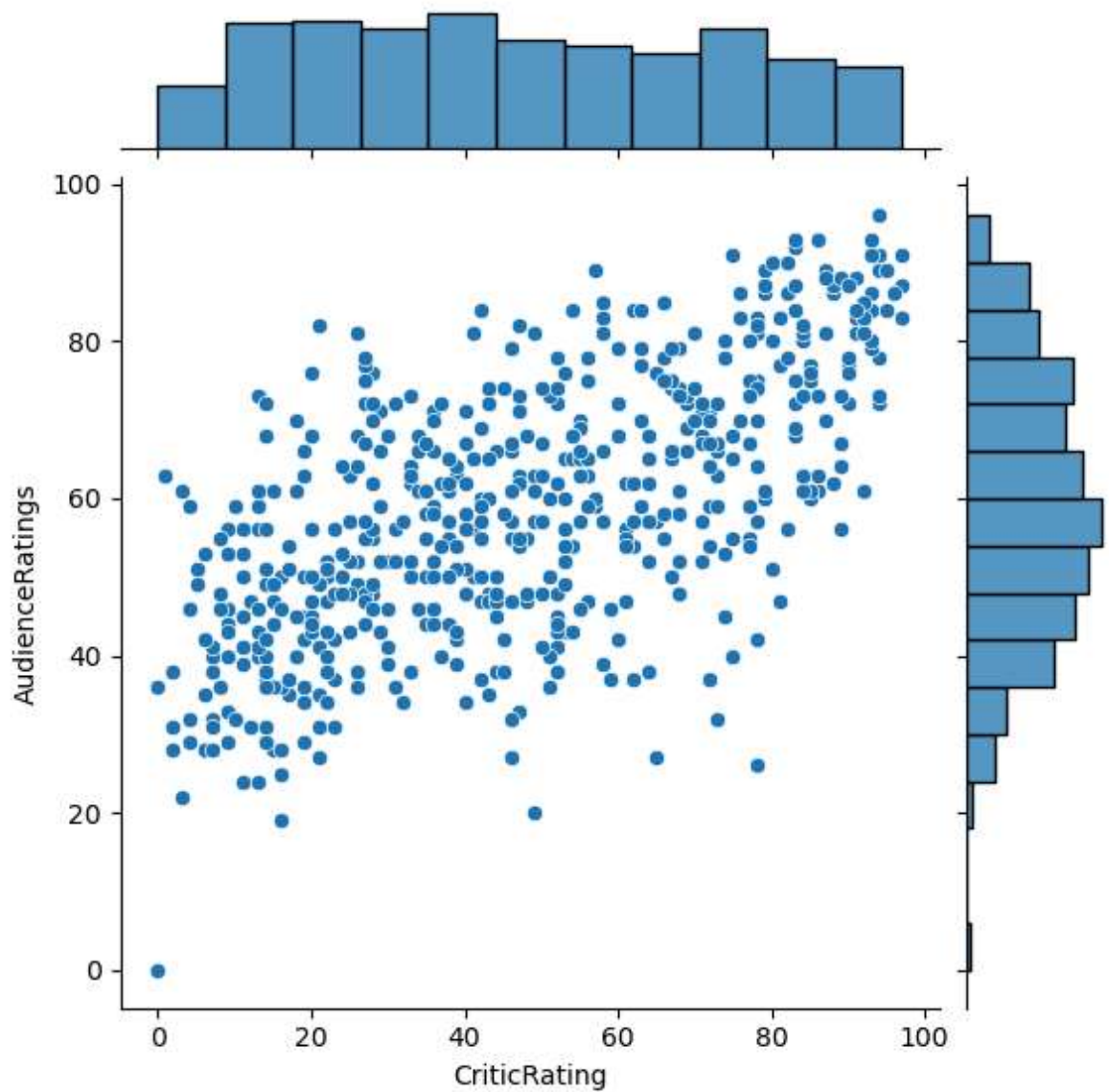
```
Out[21]:
```

|              | CriticRating | AudienceRatings | BudgetMillions |
|--------------|--------------|-----------------|----------------|
| <b>count</b> | 559.000000   | 559.000000      | 559.000000     |
| <b>mean</b>  | 47.309481    | 58.744186       | 50.236136      |
| <b>std</b>   | 26.413091    | 16.826887       | 48.731817      |
| <b>min</b>   | 0.000000     | 0.000000        | 0.000000       |
| <b>25%</b>   | 25.000000    | 47.000000       | 20.000000      |
| <b>50%</b>   | 46.000000    | 58.000000       | 35.000000      |
| <b>75%</b>   | 70.000000    | 72.000000       | 65.000000      |
| <b>max</b>   | 97.000000    | 96.000000       | 300.000000     |

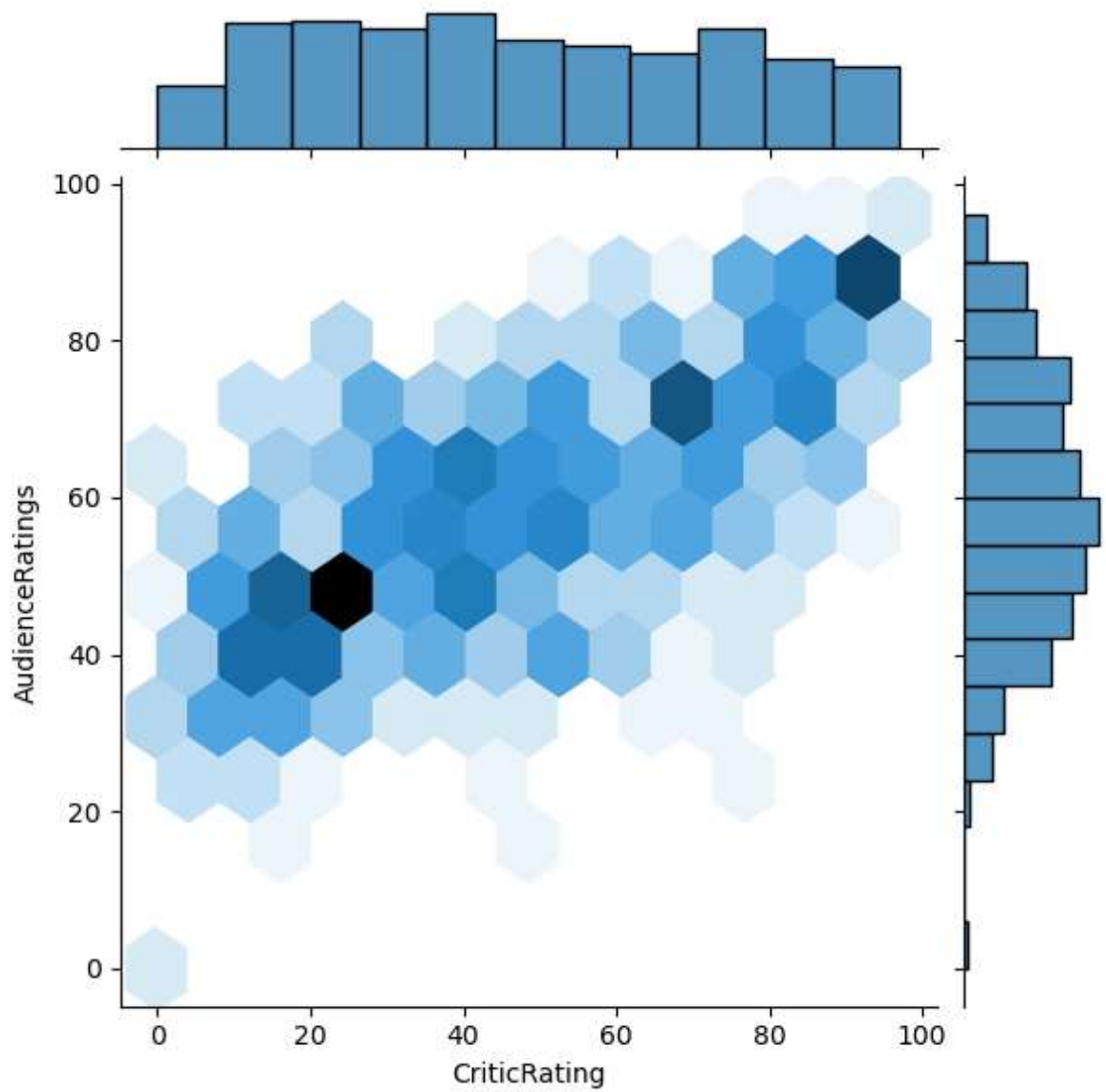
```
In [22]: from matplotlib import pyplot as plt#visualisation
import seaborn as sns#advanced visualisation

import warnings
warnings.filterwarnings('ignore')
```

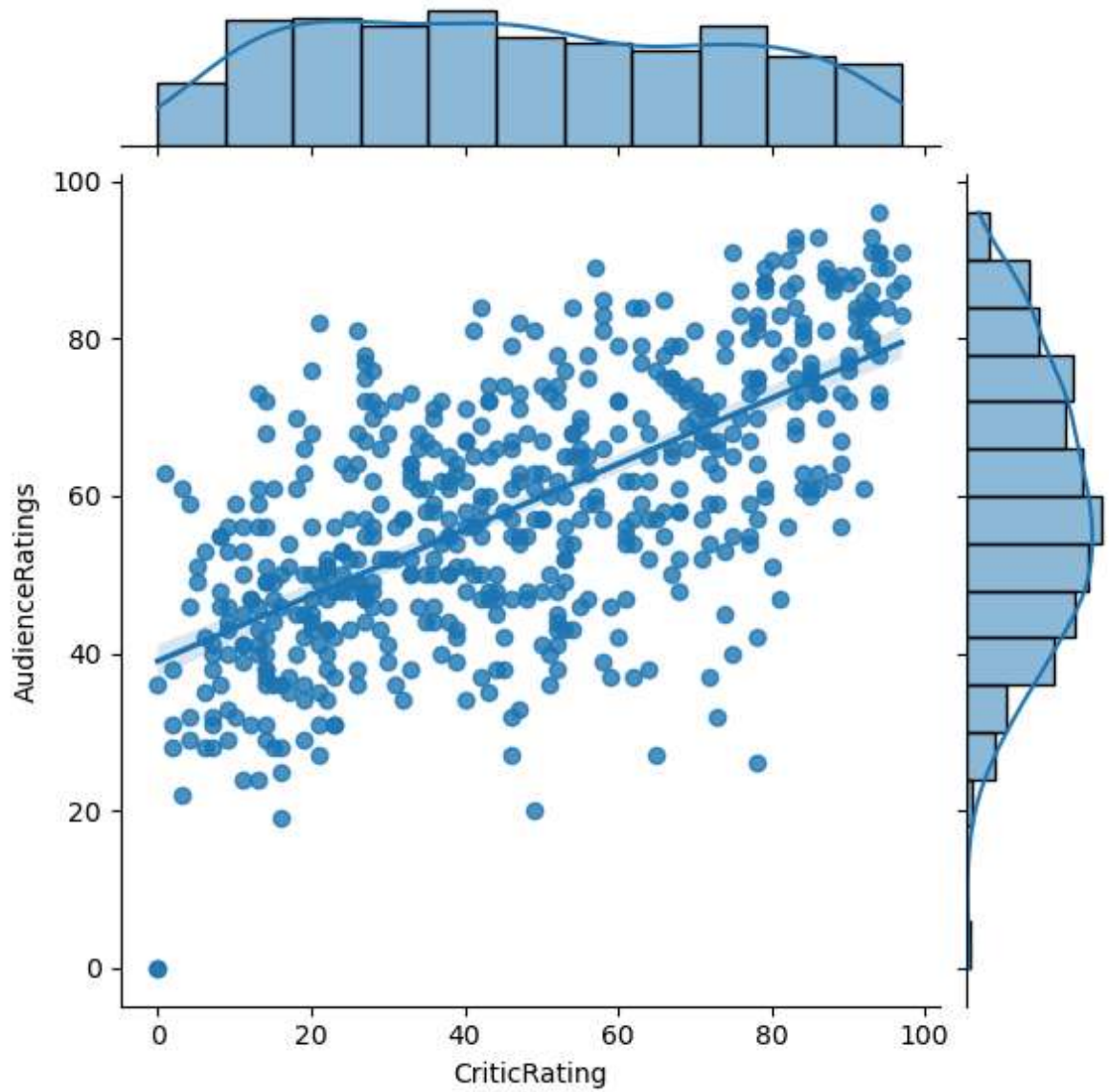
```
In [23]: j = sns.jointplot(data = mve, x = 'CriticRating', y='AudienceRatings', kind='scatt
```



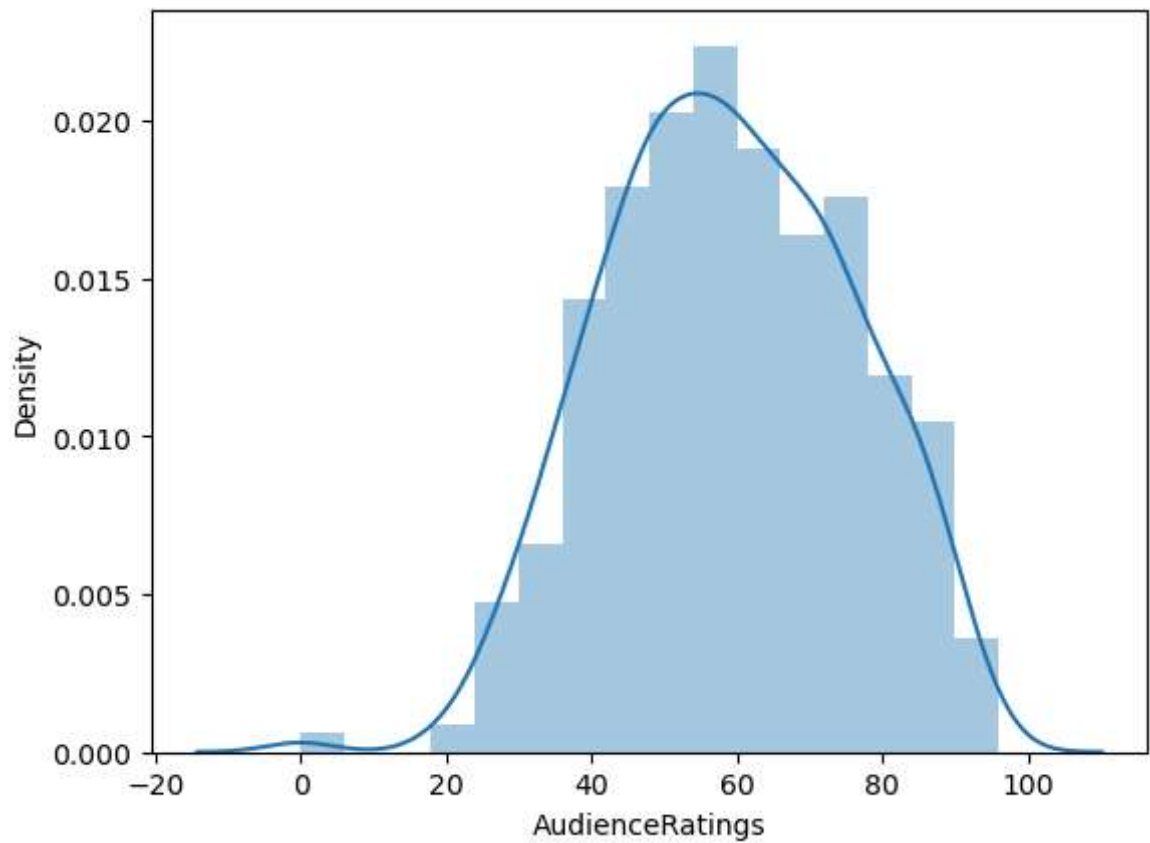
```
In [24]: j = sns.jointplot(data = mve, x = 'CriticRating', y='AudienceRatings', kind='hex')
```



```
In [25]: j = sns.jointplot(data = mve, x = 'CriticRating',y='AudienceRatings',kind='reg')
```

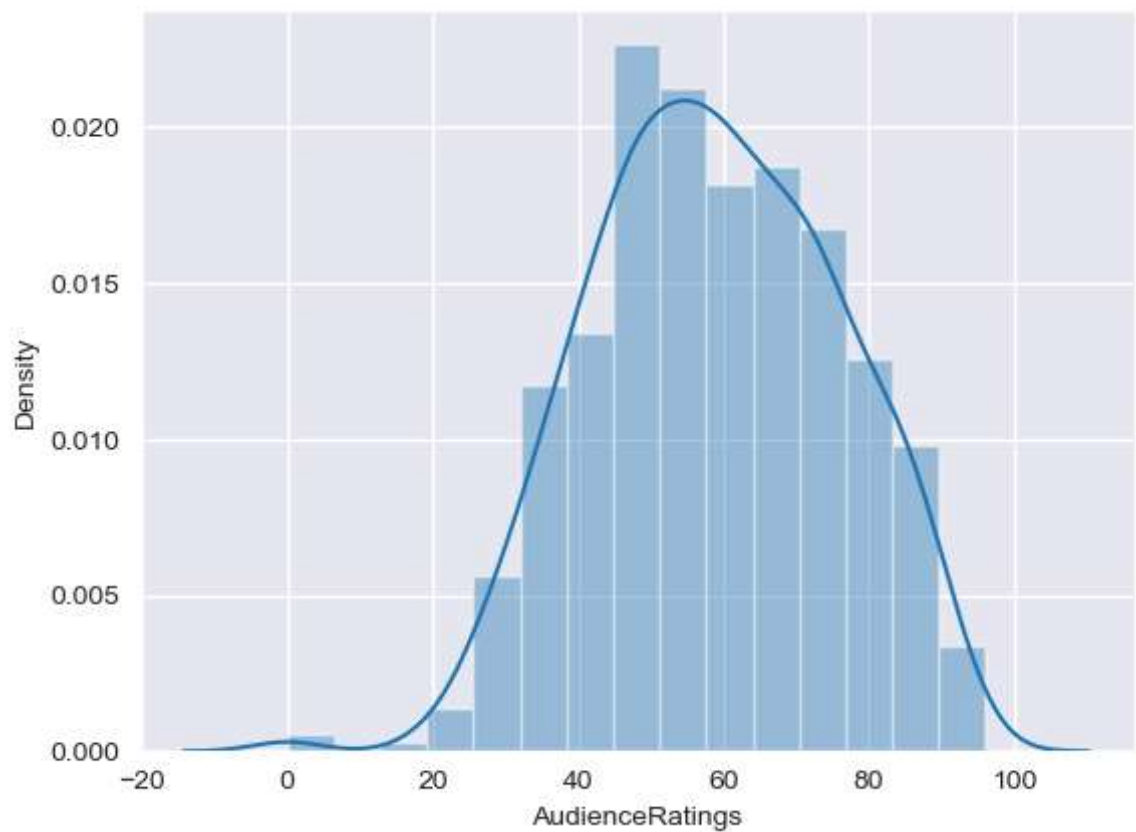


```
In [26]: m1 = sns.distplot(mve.AudienceRatings)
```



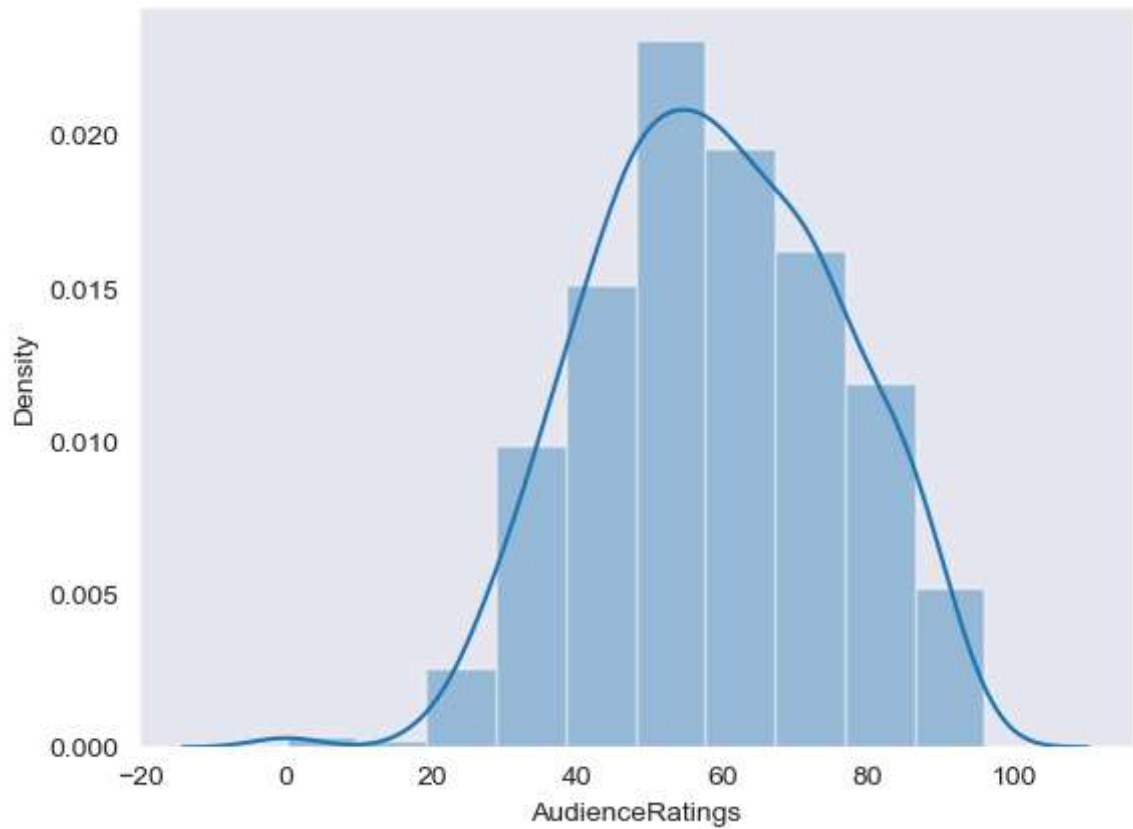
```
In [27]: sns.set_style('darkgrid')
```

```
In [28]: m2 = sns.distplot(mve.AudienceRatings, bins =15)
```

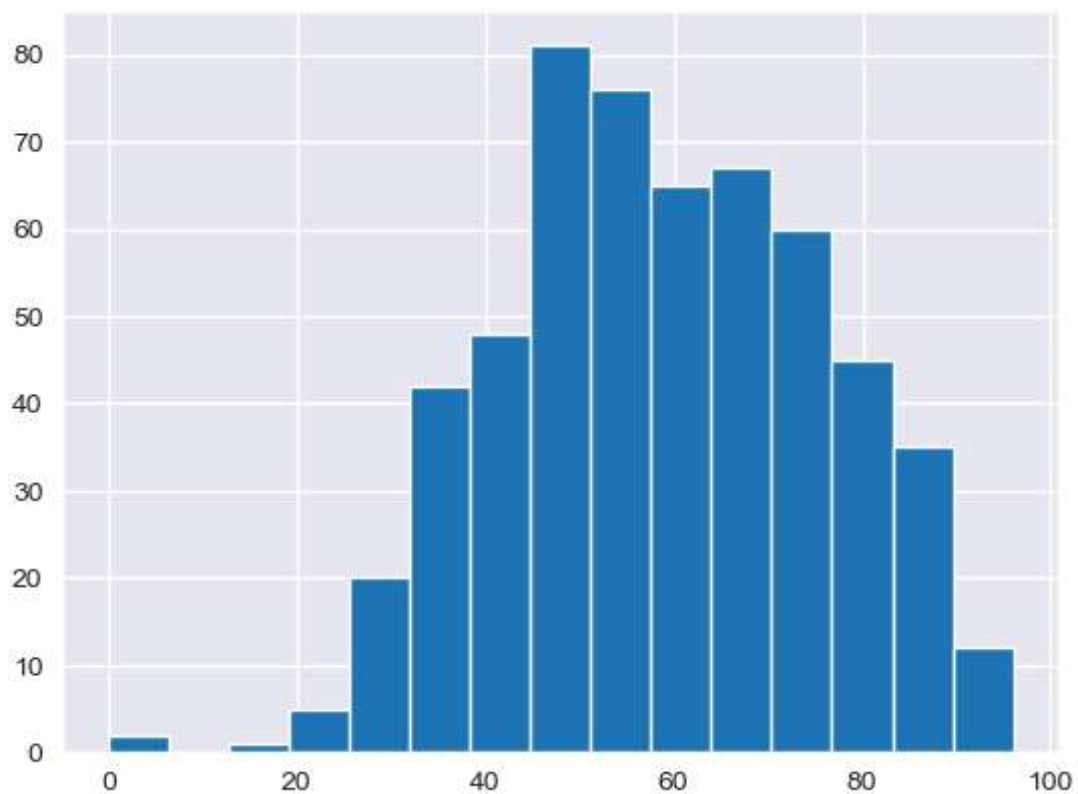


```
In [29]: sns.set_style('dark')
```

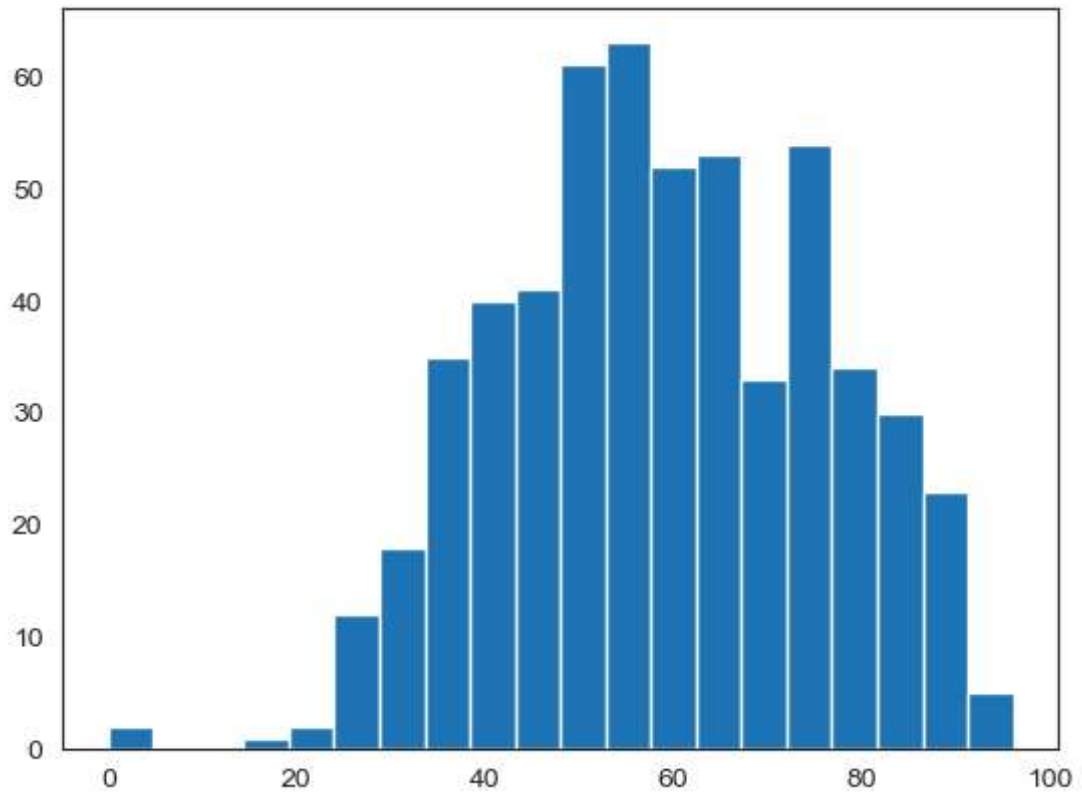
```
In [30]: m2 = sns.distplot(mve.AudienceRatings, bins =10)
```



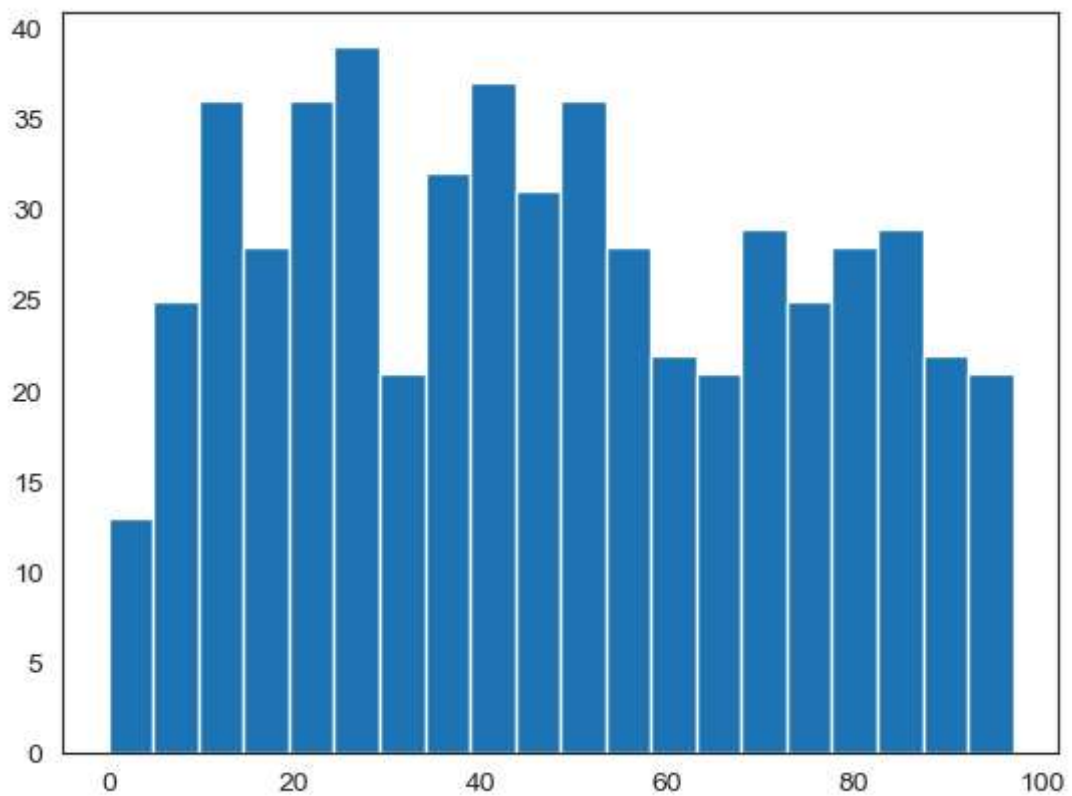
```
In [31]: sns.set_style('darkgrid')  
n1 = plt.hist(mve.AudienceRatings, bins=15)
```



```
In [32]: sns.set_style('white')  
n1 = plt.hist(mve.AudienceRatings, bins=20)#normal distribution and bell curve
```

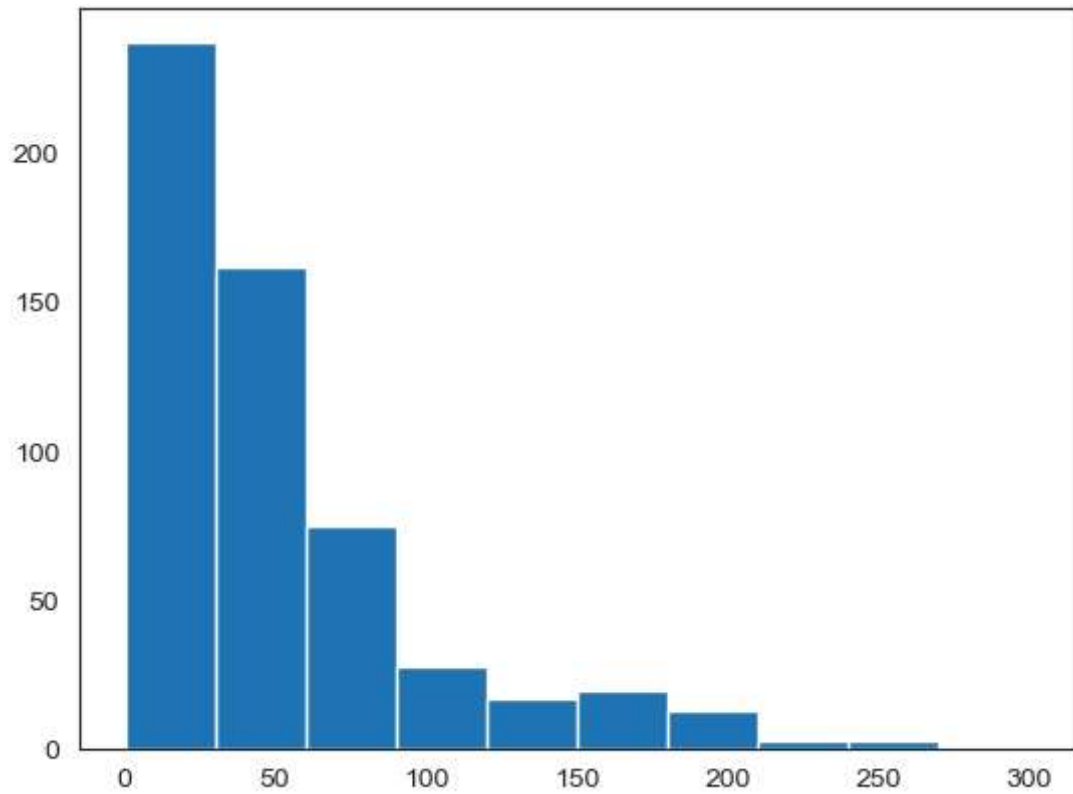


```
In [33]: n1 = plt.hist(mve.CriticRating, bins=20)#uniform distribution
```

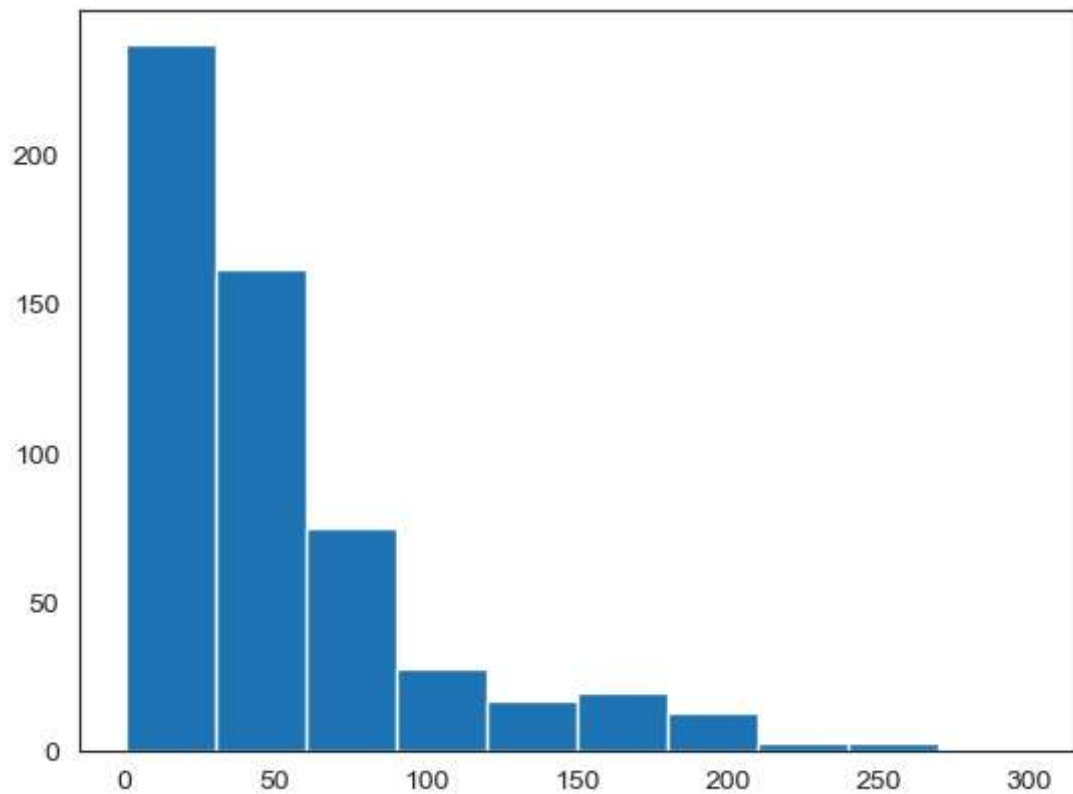


creating stacked histograms

```
In [34]: h1 = plt.hist(mve.BudgetMillions)
```



```
In [35]: plt.hist(mve.BudgetMillions)#any code works  
plt.show()
```



```
In [36]: mve.Genre
```

```

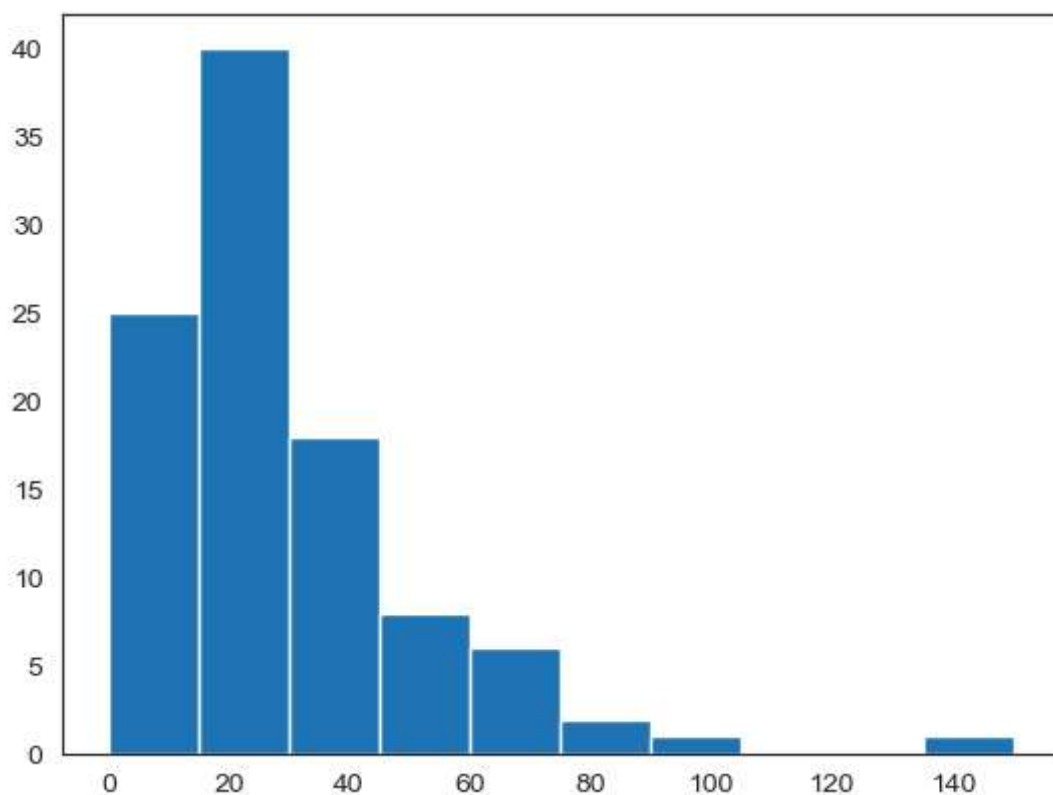
Out[36]: 0      Comedy
         1      Adventure
         2      Action
         3      Adventure
         4      Comedy
         ...
        554     Comedy
        555     Comedy
        556     Thriller
        557     Action
        558     Comedy
Name: Genre, Length: 559, dtype: category
Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance', 'Thriller']

```

```

In [37]: plt.hist(mve[mve.Genre=='Drama'].BudgetMillions)
         plt.show()

```



```

In [38]: mve.head()

```

```

Out[38]:

```

|   | Film                 | Genre     | CriticRating | AudienceRatings | BudgetMillions | Year |
|---|----------------------|-----------|--------------|-----------------|----------------|------|
| 0 | (500) Days of Summer | Comedy    | 87           | 81              | 8              | 2009 |
| 1 | 10,000 B.C.          | Adventure | 9            | 44              | 105            | 2008 |
| 2 | 12 Rounds            | Action    | 30           | 52              | 20             | 2009 |
| 3 | 127 Hours            | Adventure | 93           | 84              | 18             | 2010 |
| 4 | 17 Again             | Comedy    | 55           | 70              | 20             | 2009 |

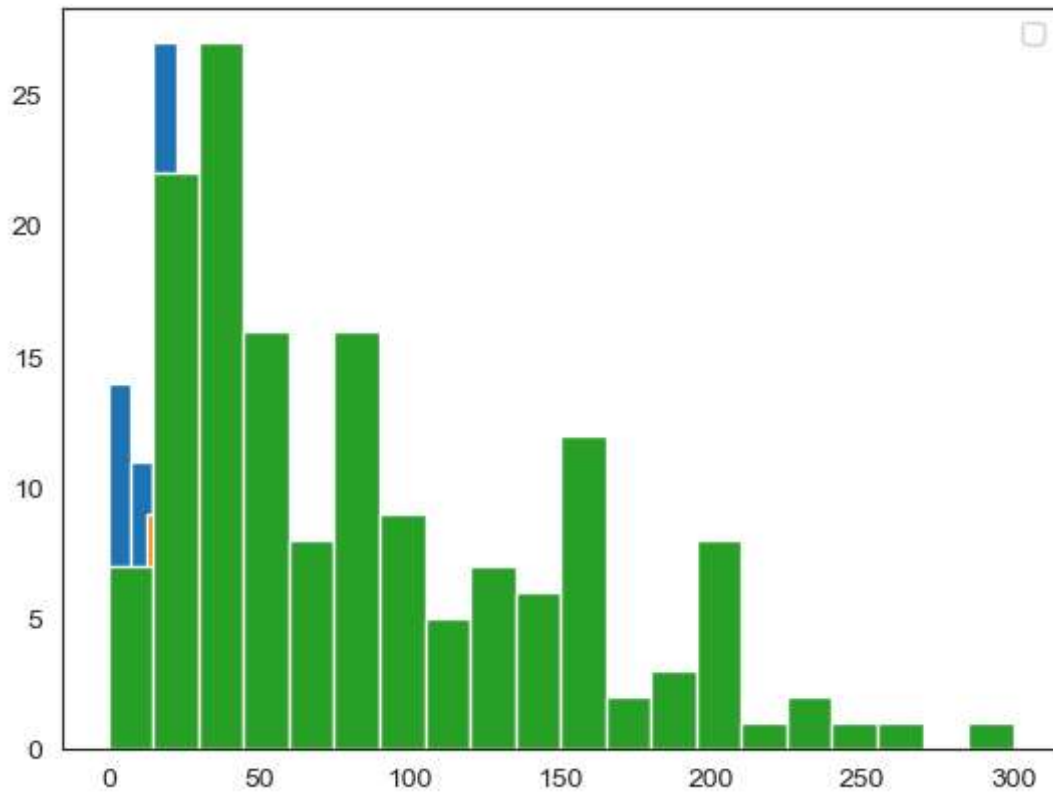
```

In [39]: mve.Genre.unique()

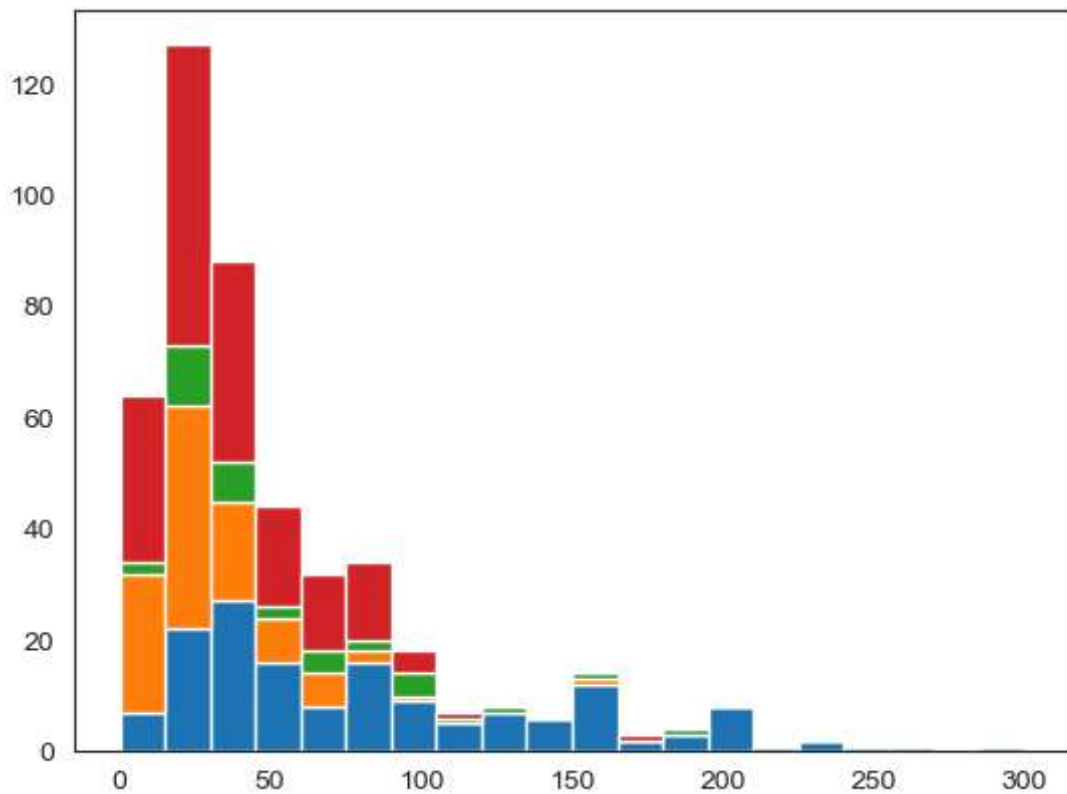
```

Out[39]: ['Comedy', 'Adventure', 'Action', 'Horror', 'Drama', 'Romance', 'Thriller']  
 Categories (7, object): ['Action', 'Adventure', 'Comedy', 'Drama', 'Horror', 'Romance', 'Thriller']

```
In [40]: plt.hist(mve[mve.Genre=='Drama'].BudgetMillions,bins=20)
plt.hist(mve[mve.Genre=='Thriller'].BudgetMillions,bins=20)
plt.hist(mve[mve.Genre=='Action'].BudgetMillions,bins=20)
plt.legend()
plt.show()
```



```
In [41]: plt.hist([mve[mve.Genre == 'Action'].BudgetMillions, \
mve[mve.Genre == 'Drama'].BudgetMillions, \
mve[mve.Genre == 'Thriller'].BudgetMillions, \
mve[mve.Genre == 'Comedy'].BudgetMillions],
bins = 20, stacked = True)
plt.show()
```

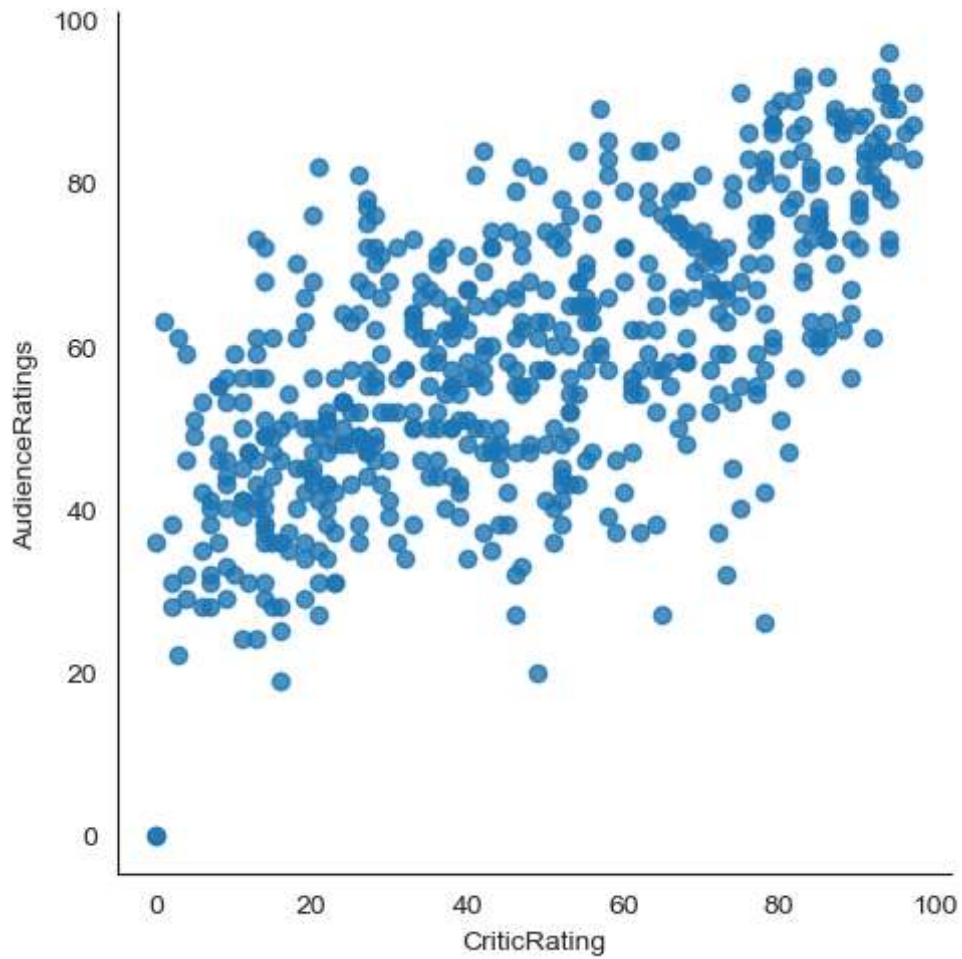


In [42]: *#since we have more than 10 genre we couldn't copy all of those and type it so w*

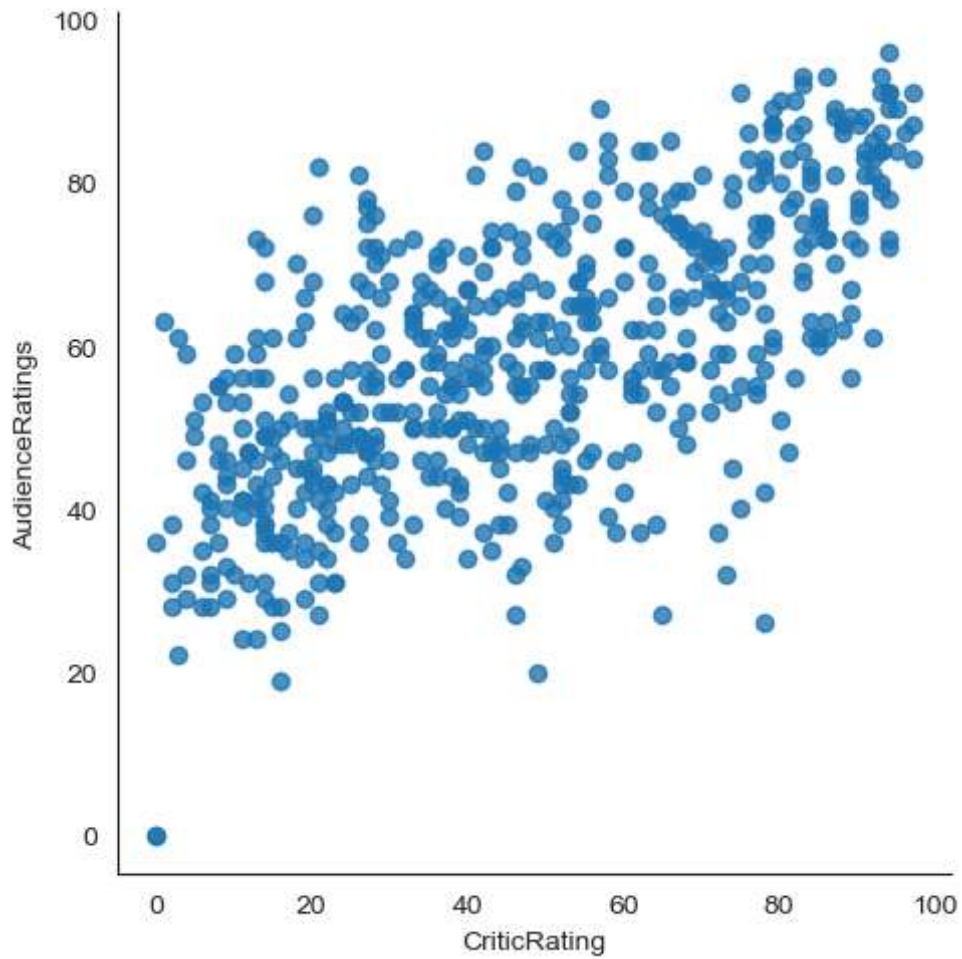
```
for gen in mve.Genre.cat.categories:  
    print(gen)
```

Action  
Adventure  
Comedy  
Drama  
Horror  
Romance  
Thriller

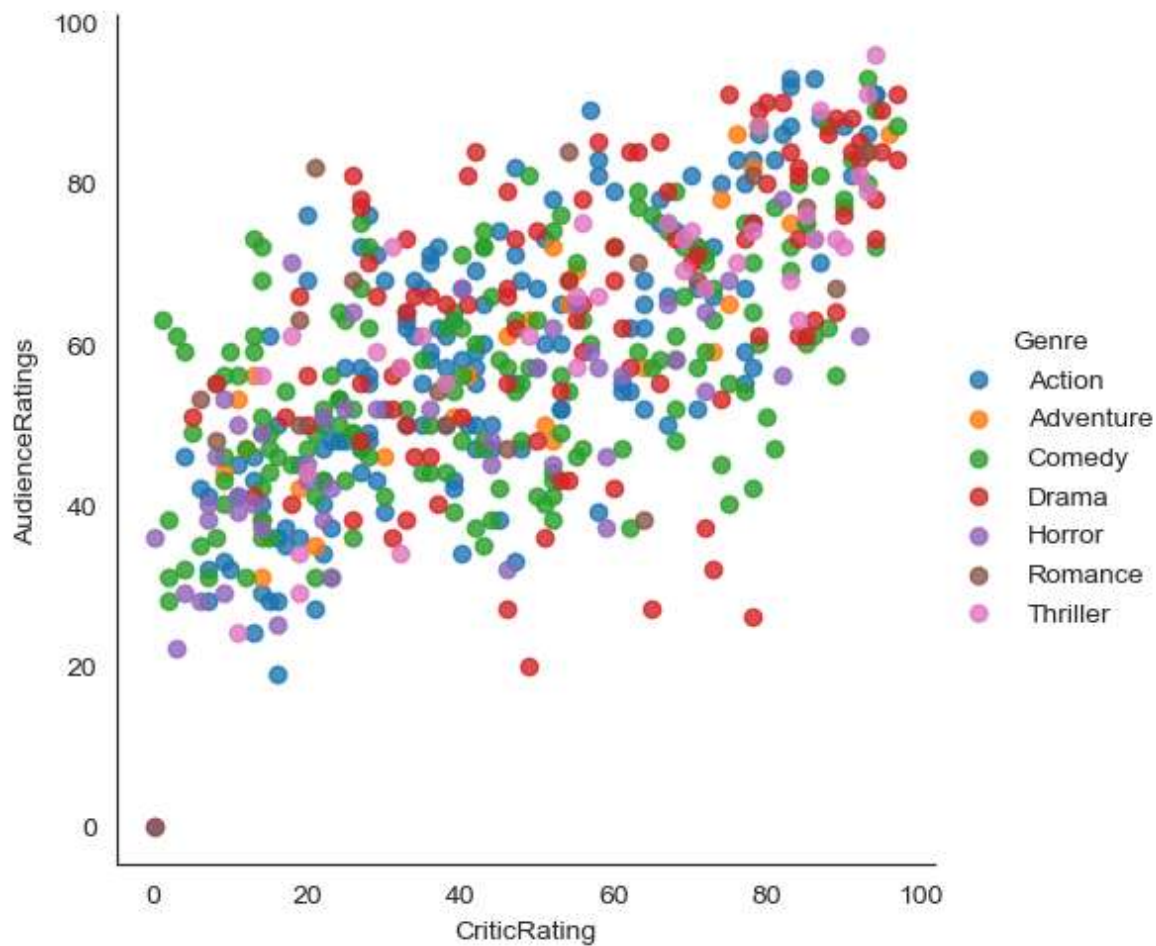
```
In [43]: vis1 = sns.lmplot(data = mve,x='CriticRating',y='AudienceRatings',\  
                           fit_reg=False)
```



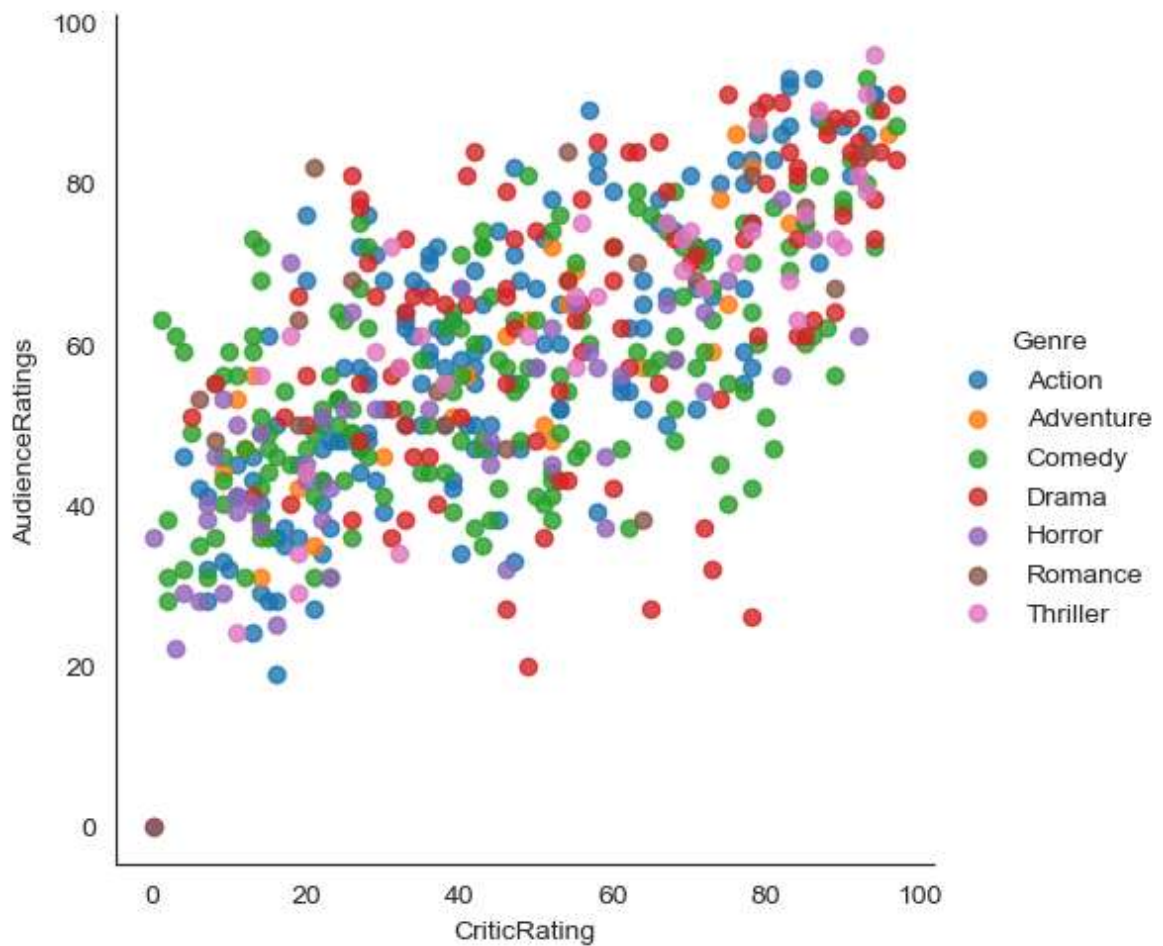
```
In [44]: vis1 = sns.lmplot(data = mve,x='CriticRating',y='AudienceRatings',fit_reg=False)
```



```
In [45]: vis1 = sns.lmplot(data = mve,x='CriticRating',y='AudienceRatings',fit_reg=False,
```

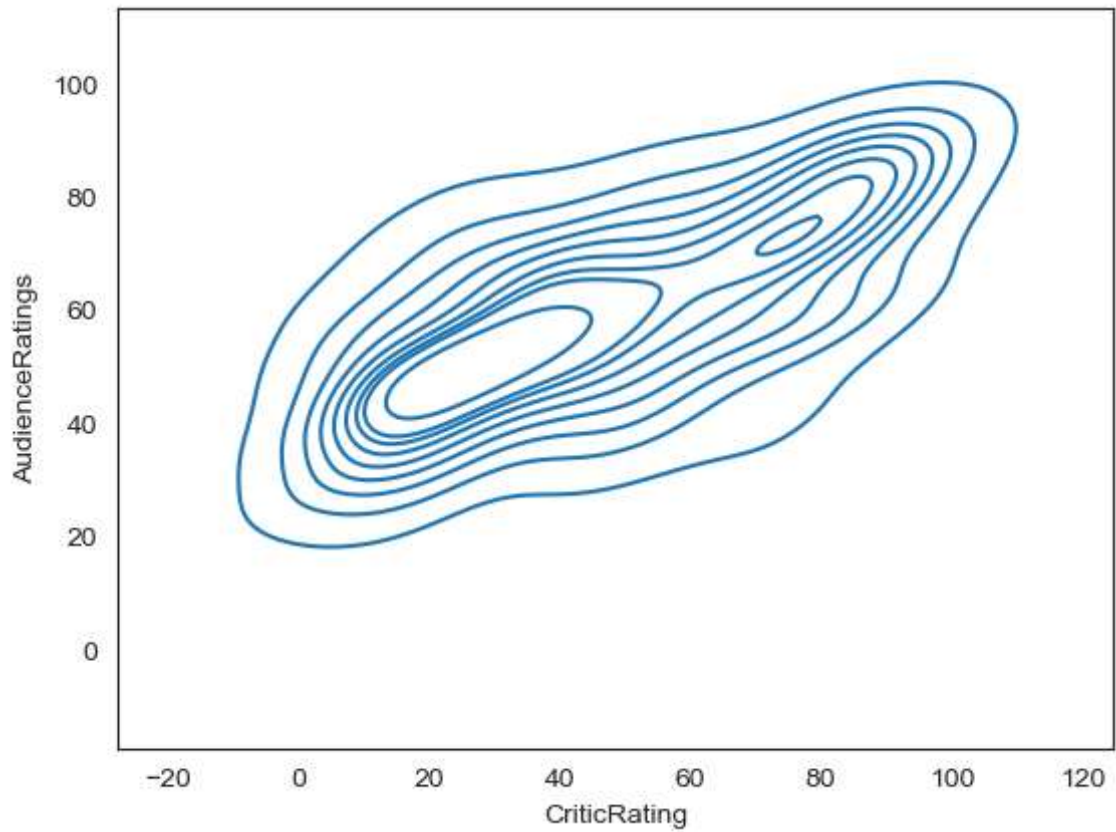


```
In [46]: vis1 = sns.lmplot(data = mve,x='CriticRating',y='AudienceRatings',\
                           fit_reg=False,hue='Genre',aspect=1)
```

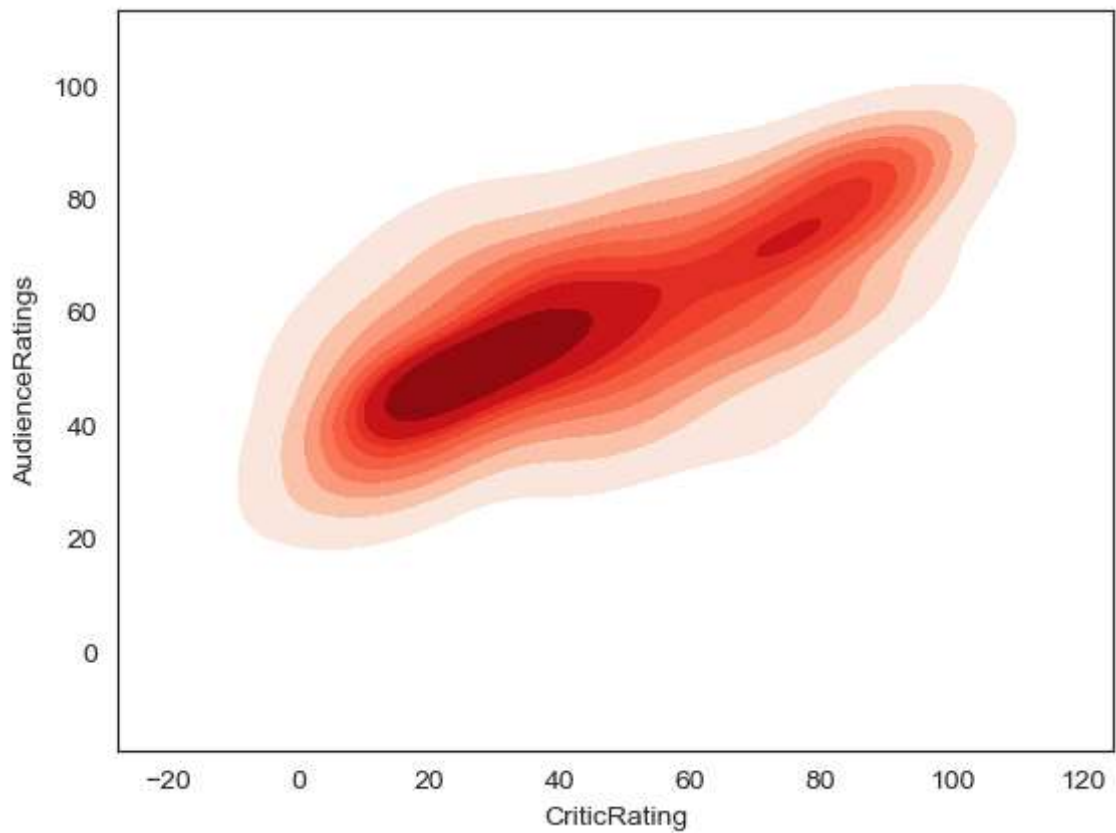


## KDE(Kernal density estimate)plot

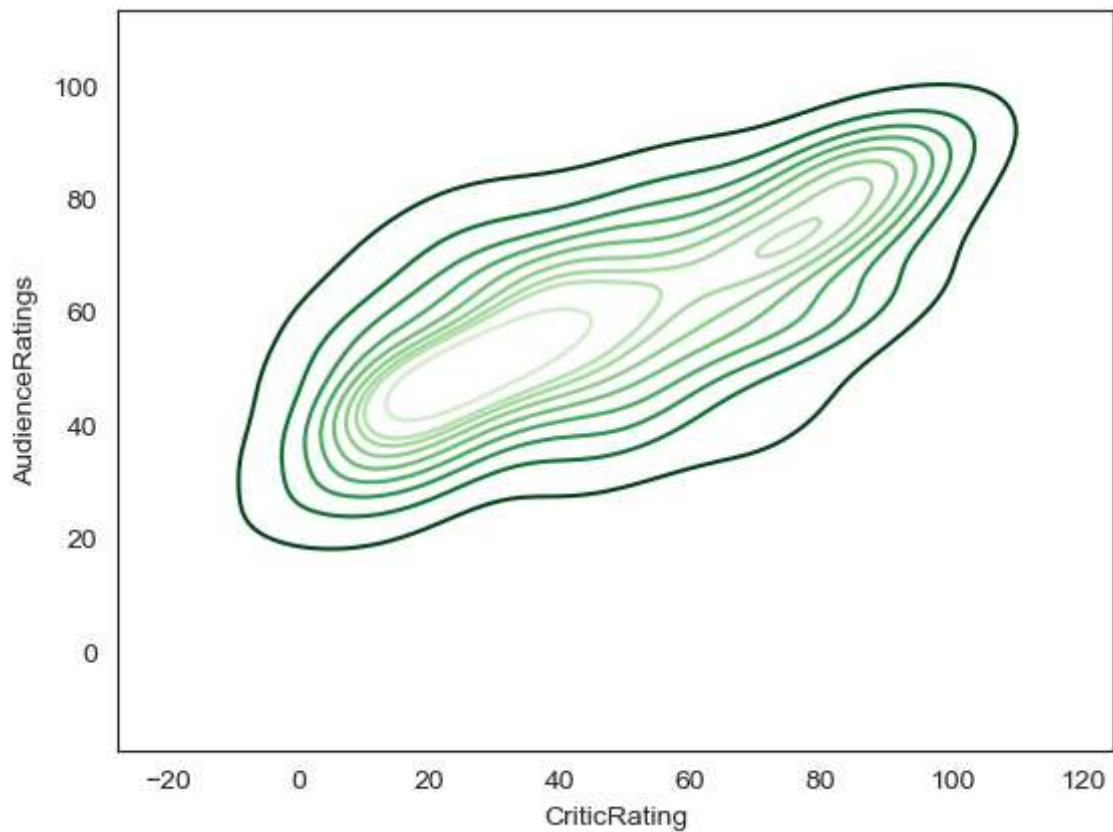
```
In [47]: k1 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings)
```



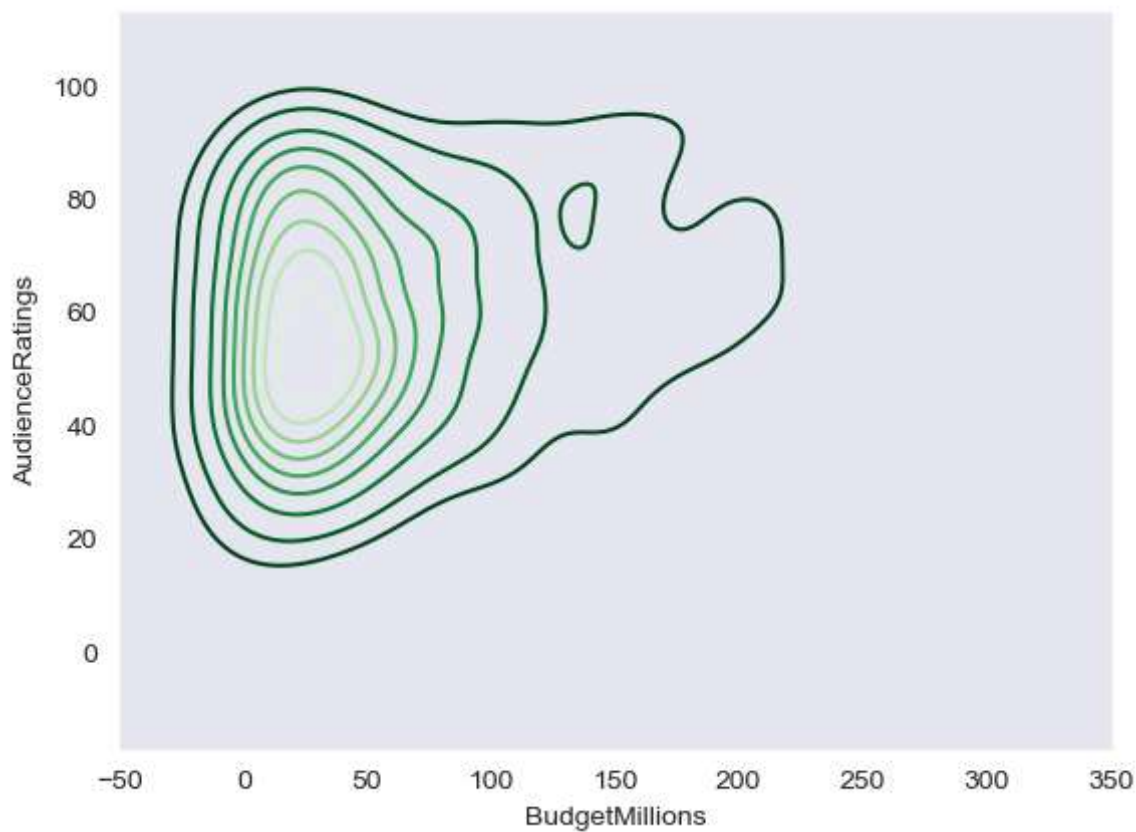
```
In [48]: k1 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings,shade=True,shade_lowes
```



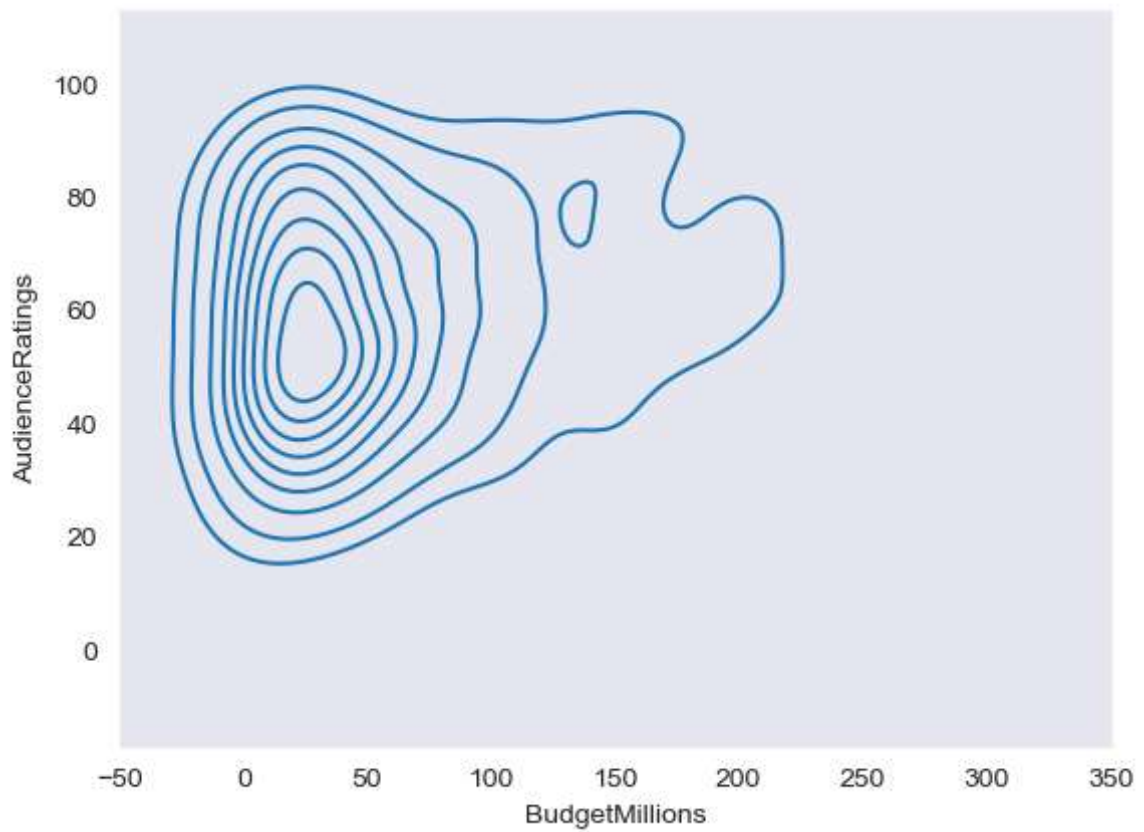
```
In [49]: k1 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings,shade_lowest=False,cma
```



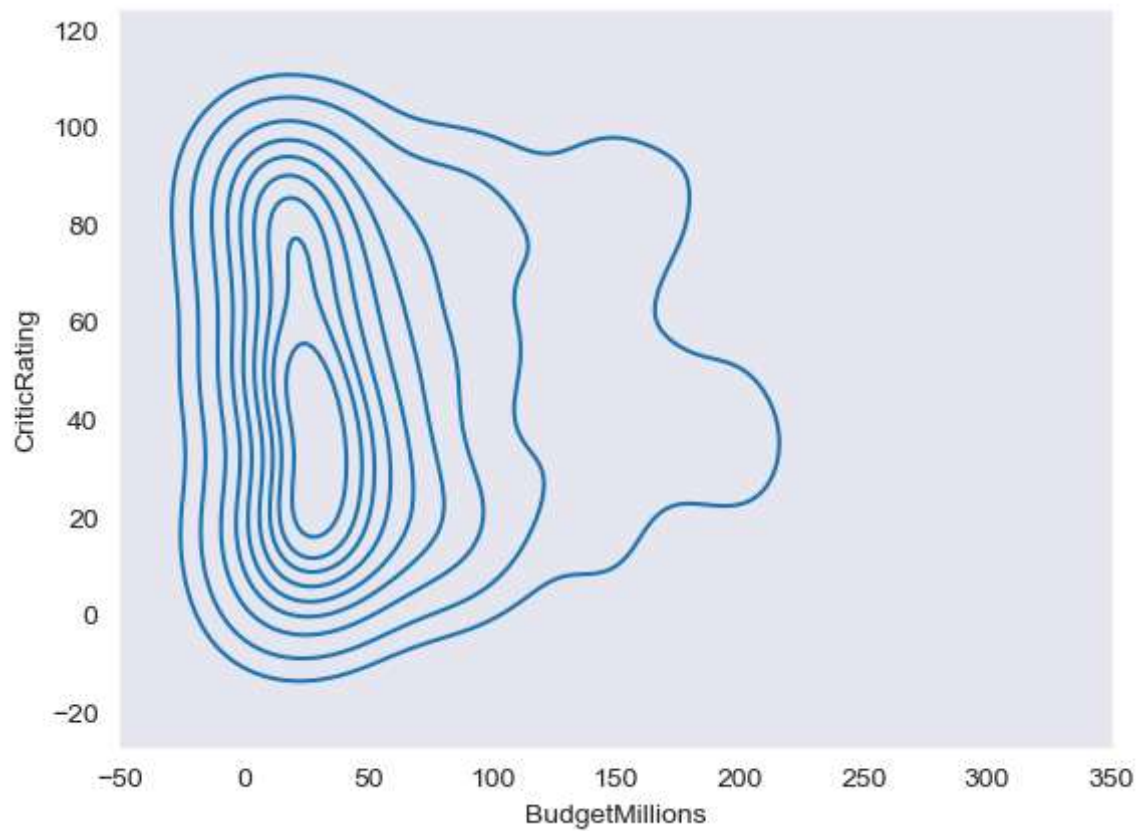
```
In [50]: sns.set_style('dark')
k1 = sns.kdeplot(x=mve.BudgetMillions,y=mve.AudienceRatings,shade_lowest=False,c
```



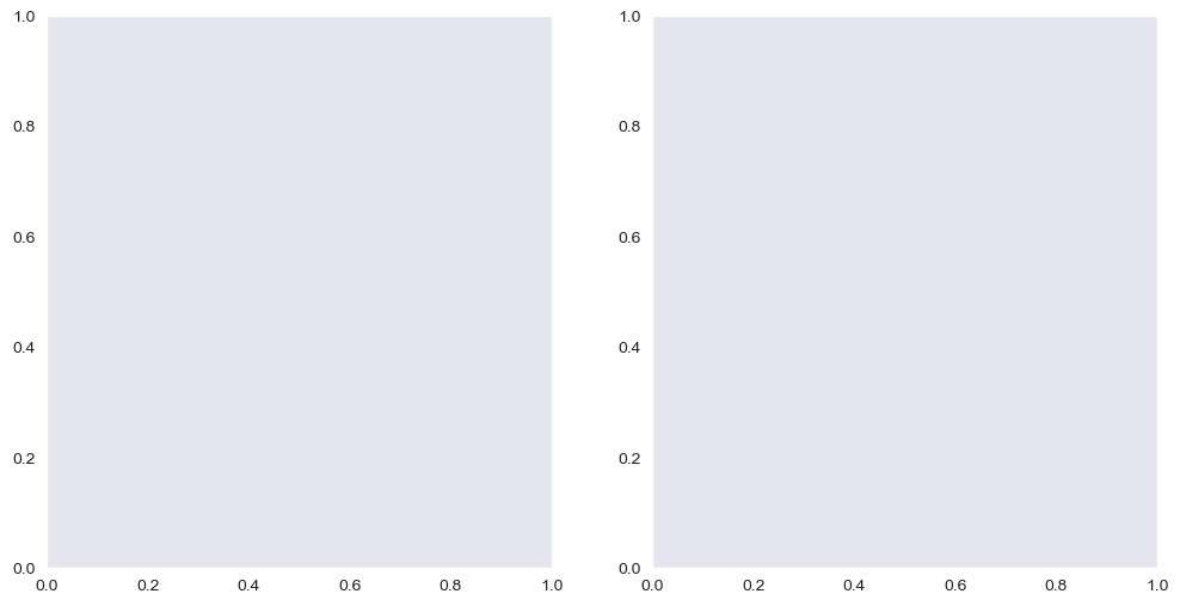
```
In [51]: sns.set_style('dark')
k1 = sns.kdeplot(x=mve.BudgetMillions,y=mve.AudienceRatings)
```



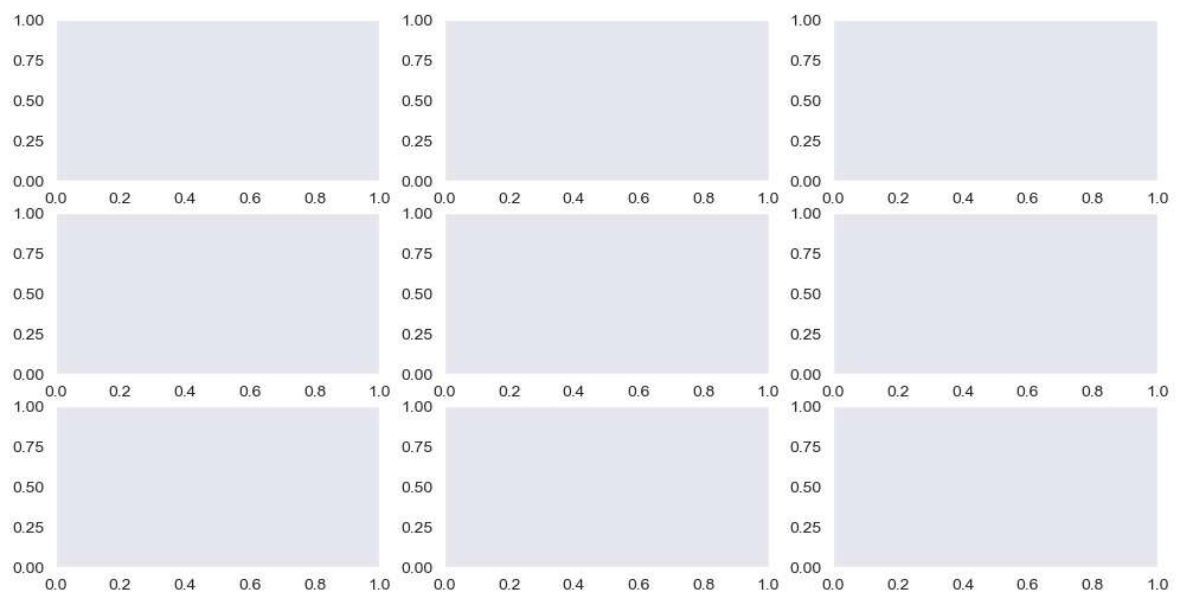
```
In [52]: k2 = sns.kdeplot(x=mve.BudgetMillions,y=mve.CriticRating)
```



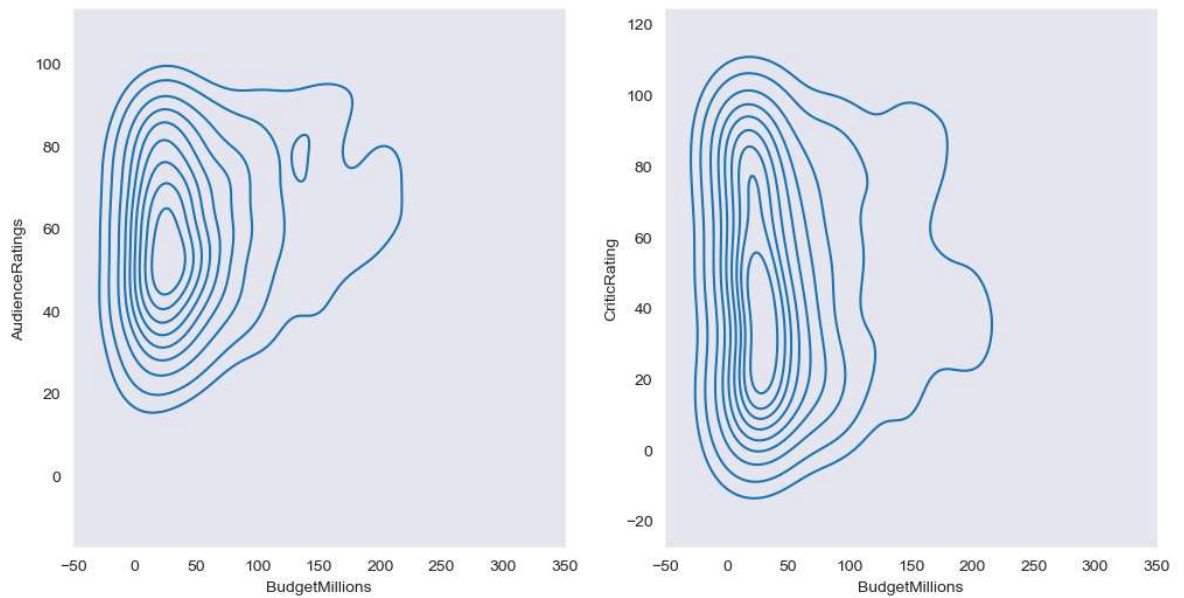
```
In [53]: f, ax = plt.subplots(1,2, figsize =(12,6))
```



```
In [54]: f, ax = plt.subplots(3,3, figsize =(12,6))
```



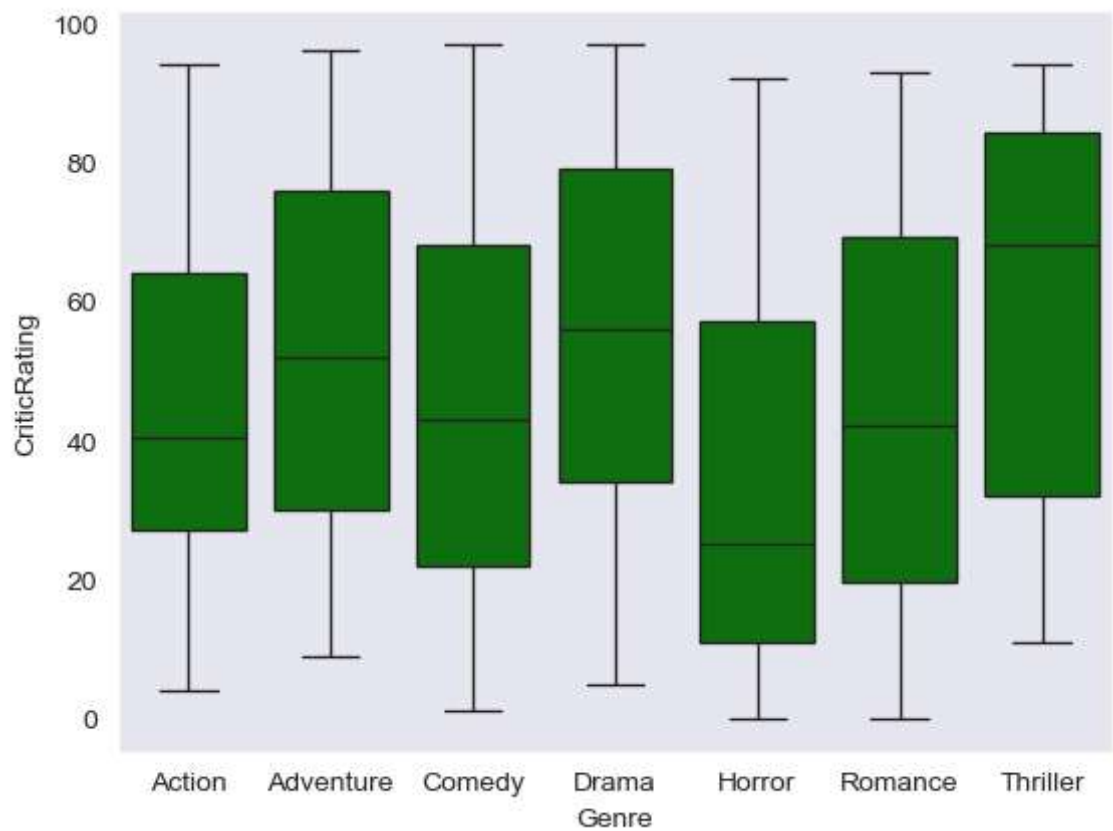
```
In [55]: f, ax = plt.subplots(1,2, figsize =(12,6))  
k1 = sns.kdeplot(x=mve.BudgetMillions,y=mve.AudienceRatings,ax=ax[0])  
k2 = sns.kdeplot(x=mve.BudgetMillions,y=mve.CriticRating,ax=ax[1])
```



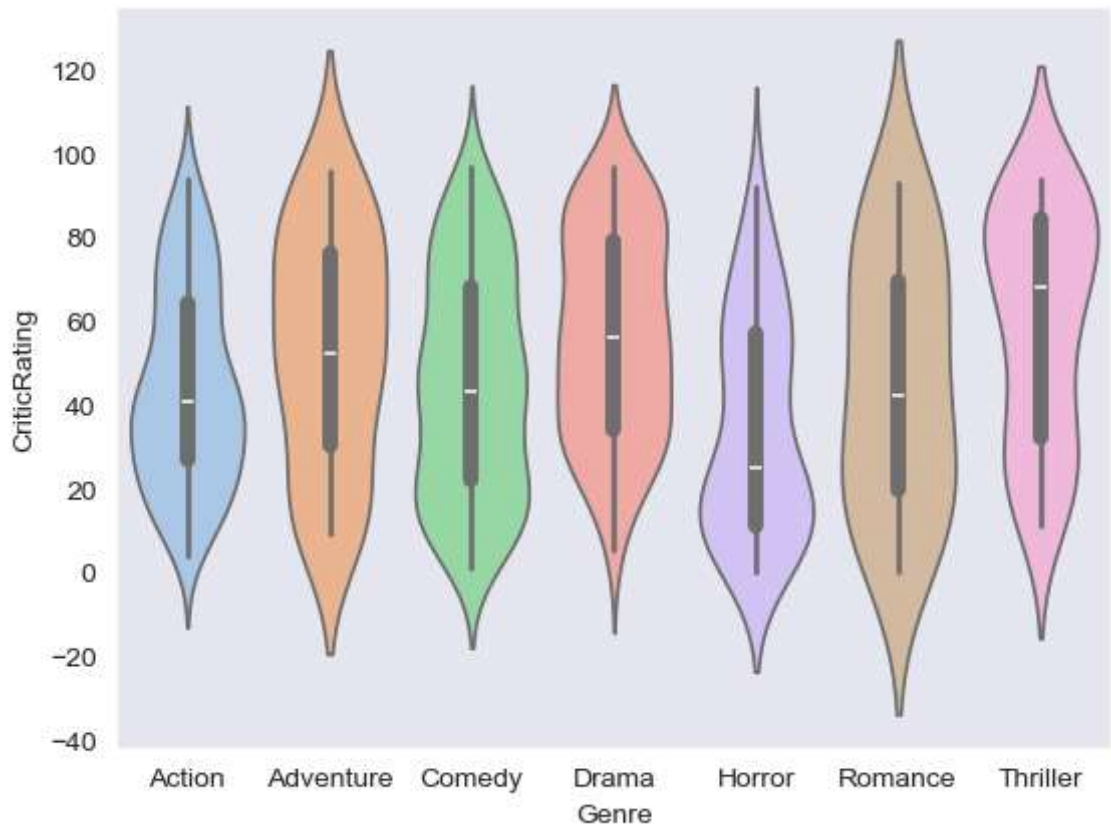
In [56]: ax

Out[56]: array([<Axes: xlabel='BudgetMillions', ylabel='AudienceRatings'>,  
 <Axes: xlabel='BudgetMillions', ylabel='CriticRating'>],  
 dtype=object)

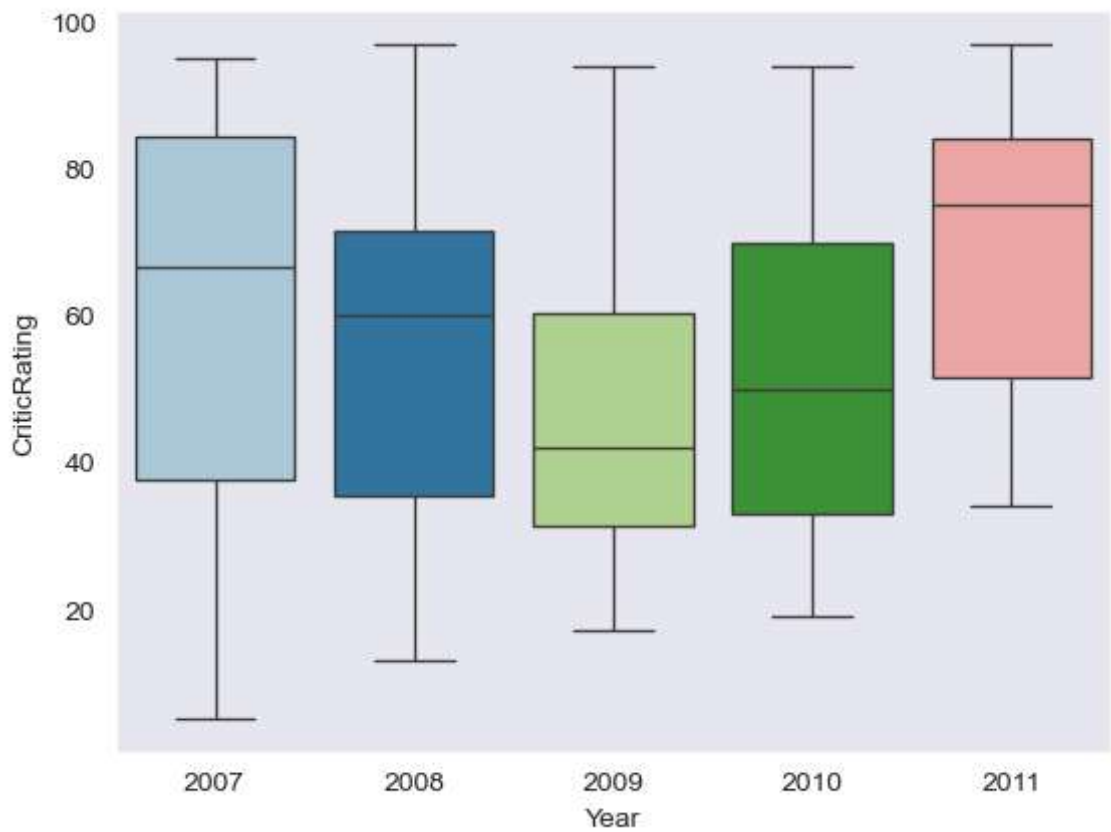
In [57]: `# box plot`  
`b = sns.boxplot(data=mve,x='Genre',y='CriticRating',color='green')`



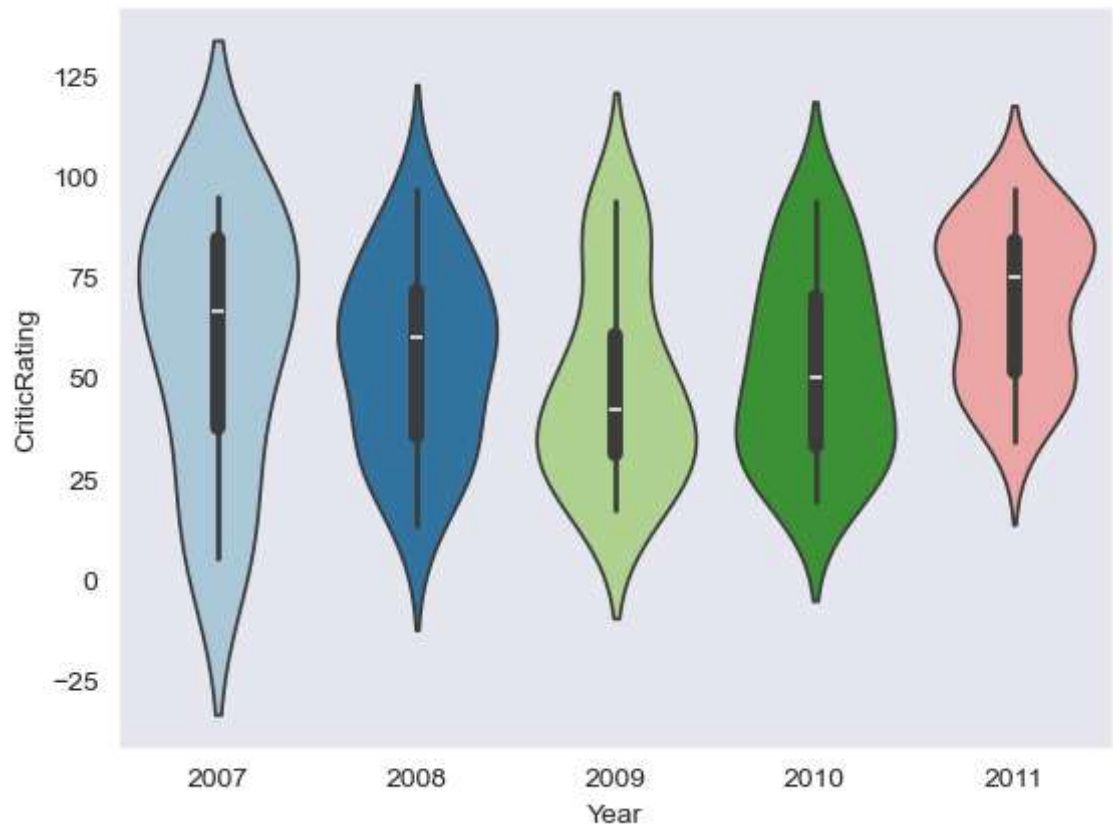
In [58]: `# violin plot`  
`v = sns.violinplot(data=mve,x='Genre',y='CriticRating',palette='pastel')`



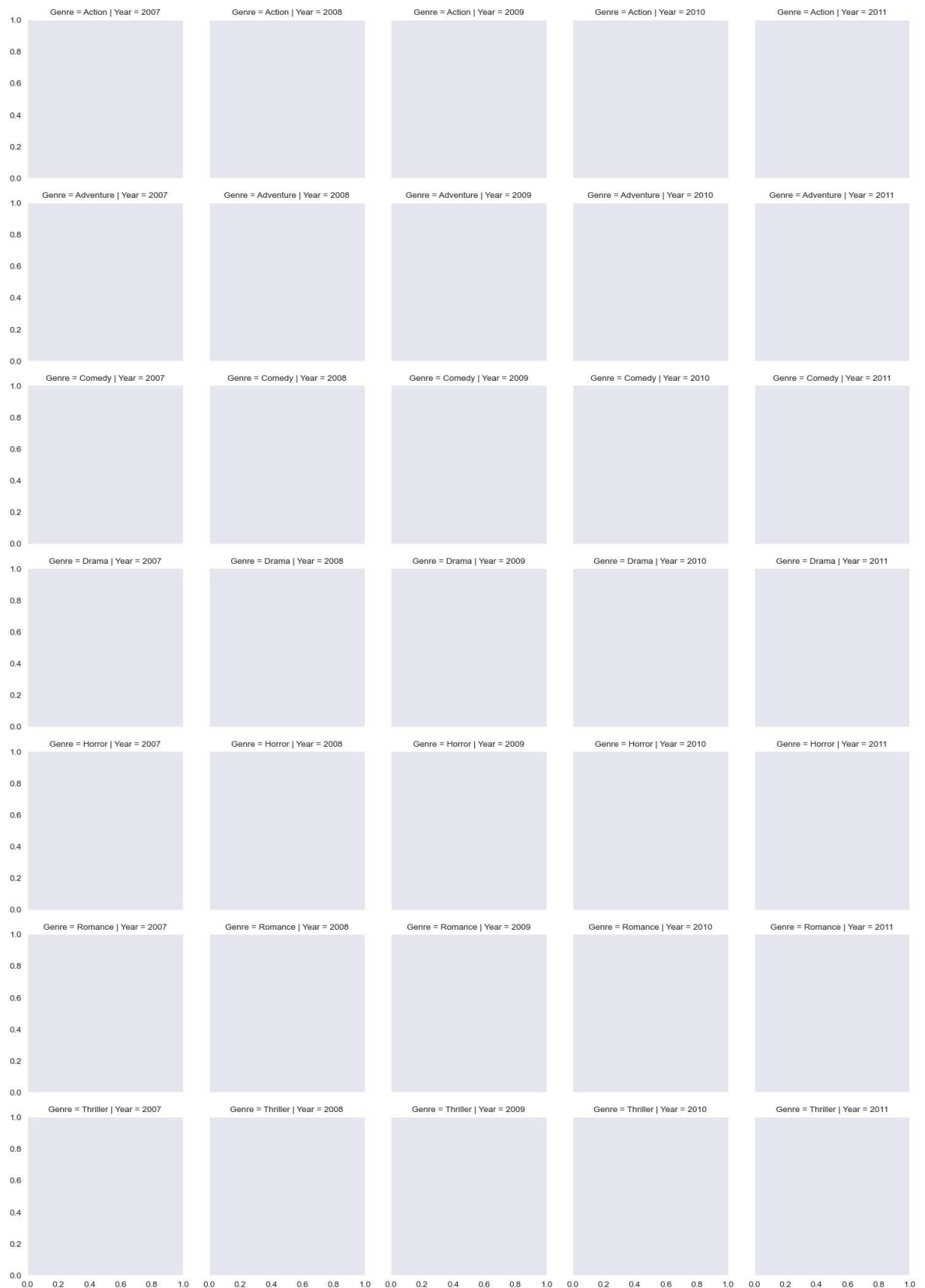
```
In [59]: w1 = sns.boxplot(data=mve[mve.Genre == 'Drama'], x='Year', y = 'CriticRating', palette='magma')
```



```
In [60]: w1 = sns.violinplot(data=mve[mve.Genre == 'Drama'], x='Year', y = 'CriticRating', palette='magma')
```

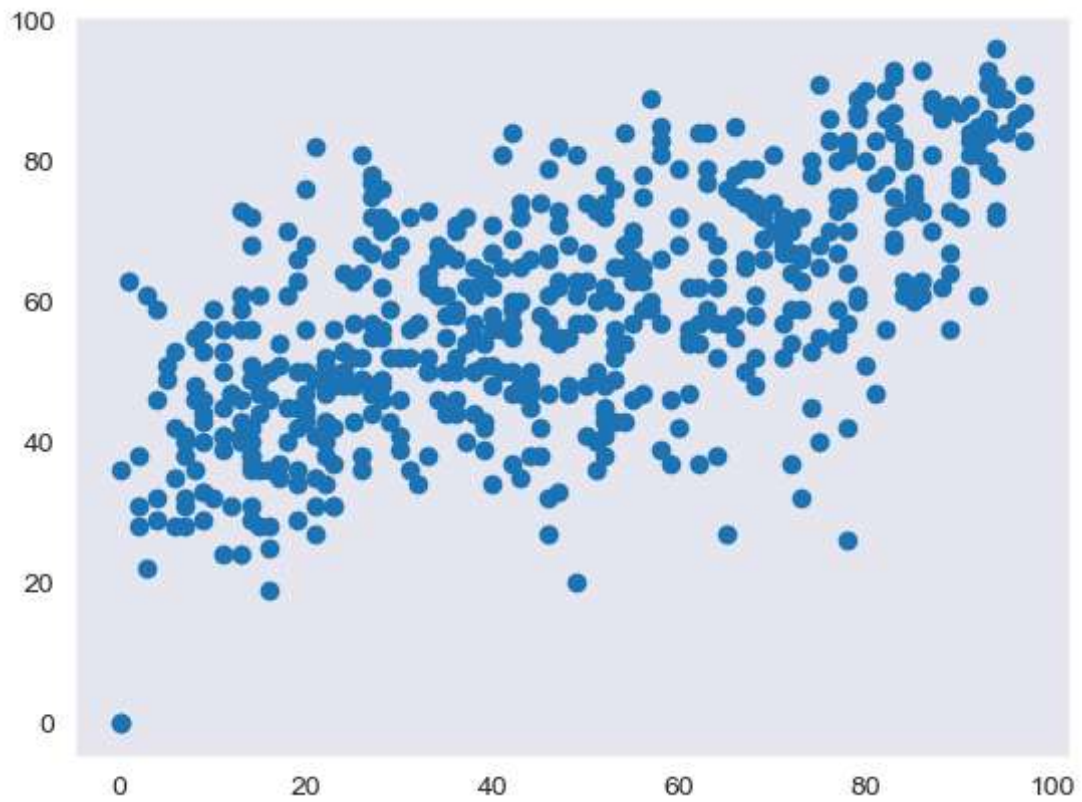


```
In [61]: # Facet grid creation
g = sns.FacetGrid (mve, row = 'Genre', col = 'Year', hue = 'Genre') #kind of subp
```

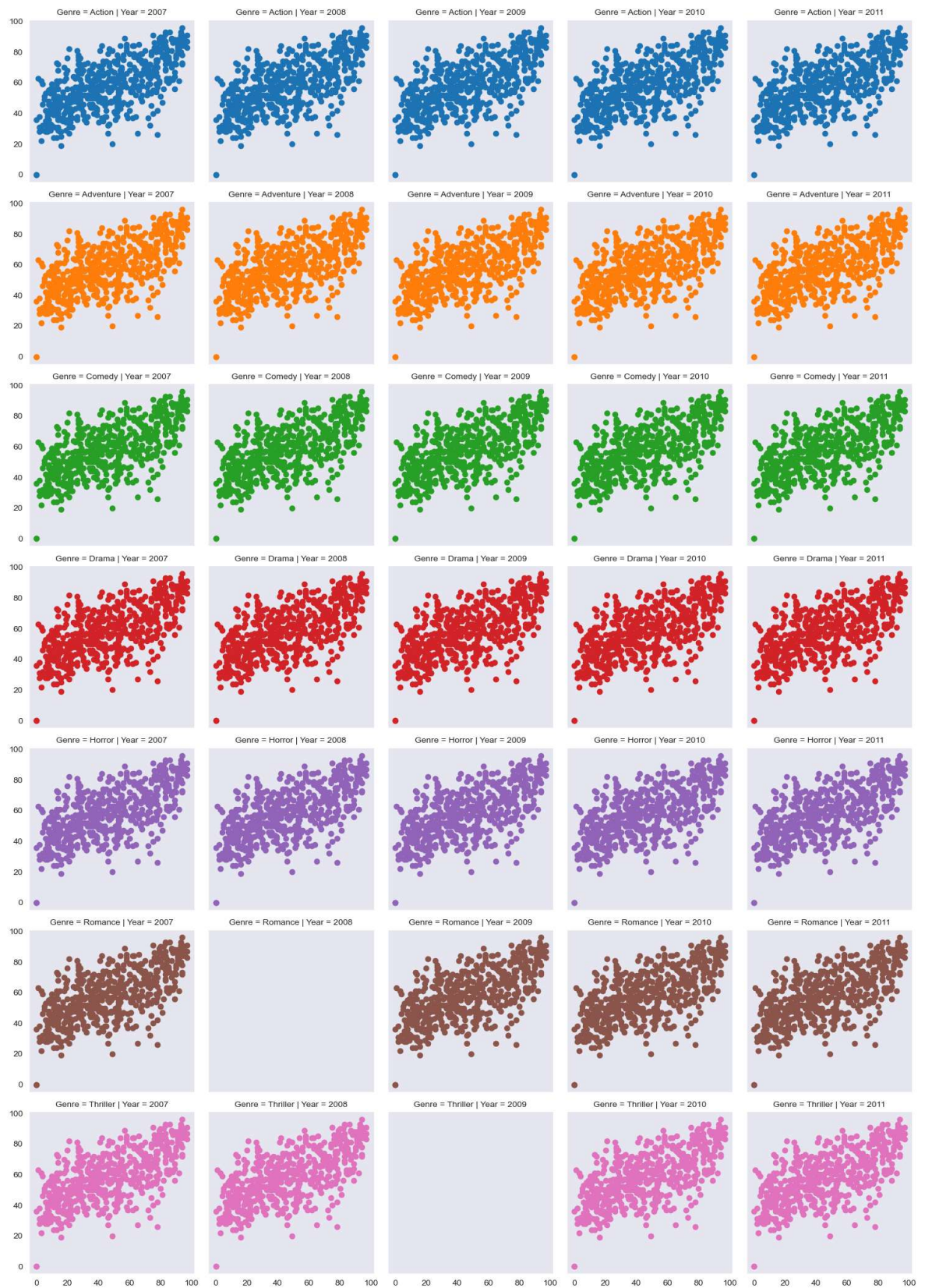


```
In [62]: plt.scatter(x=mve.CriticRating,y=mve.AudienceRatings)
```

```
Out[62]: <matplotlib.collections.PathCollection at 0x231ff0765d0>
```



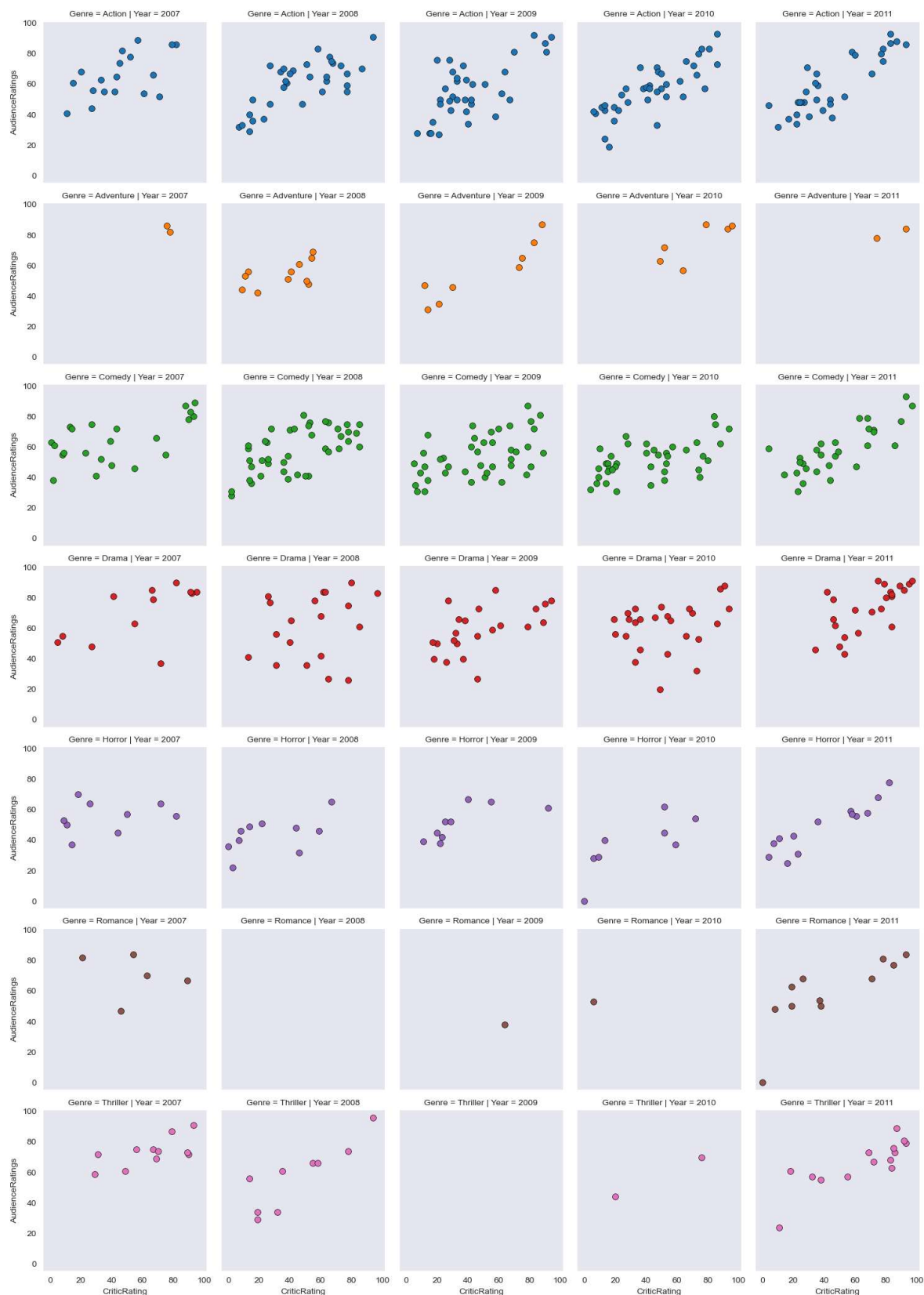
```
In [68]: g = sns.FacetGrid (mve, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.scatter, x=mve.CriticRating, y=mve.AudienceRatings)
```



```
In [70]: g = sns.FacetGrid (mve, row = 'Genre', col = 'Year', hue = 'Genre')
g = g.map(plt.hist, 'BudgetMillions')
```



```
In [74]: # scatterplot mapped in facetgrid
g = sns.FacetGrid (mve, row = 'Genre', col = 'Year', hue = 'Genre')
kws=dict(s=50, linewidth=0.5,edgecolor='black')
g=g.map(plt.scatter,'CriticRating', 'AudienceRatings',**kws)
```



```
In [80]: # dashboard-combination of charts
sns.set_style('darkgrid')
f, axes = plt.subplots(2,2, figsize = (15,15))

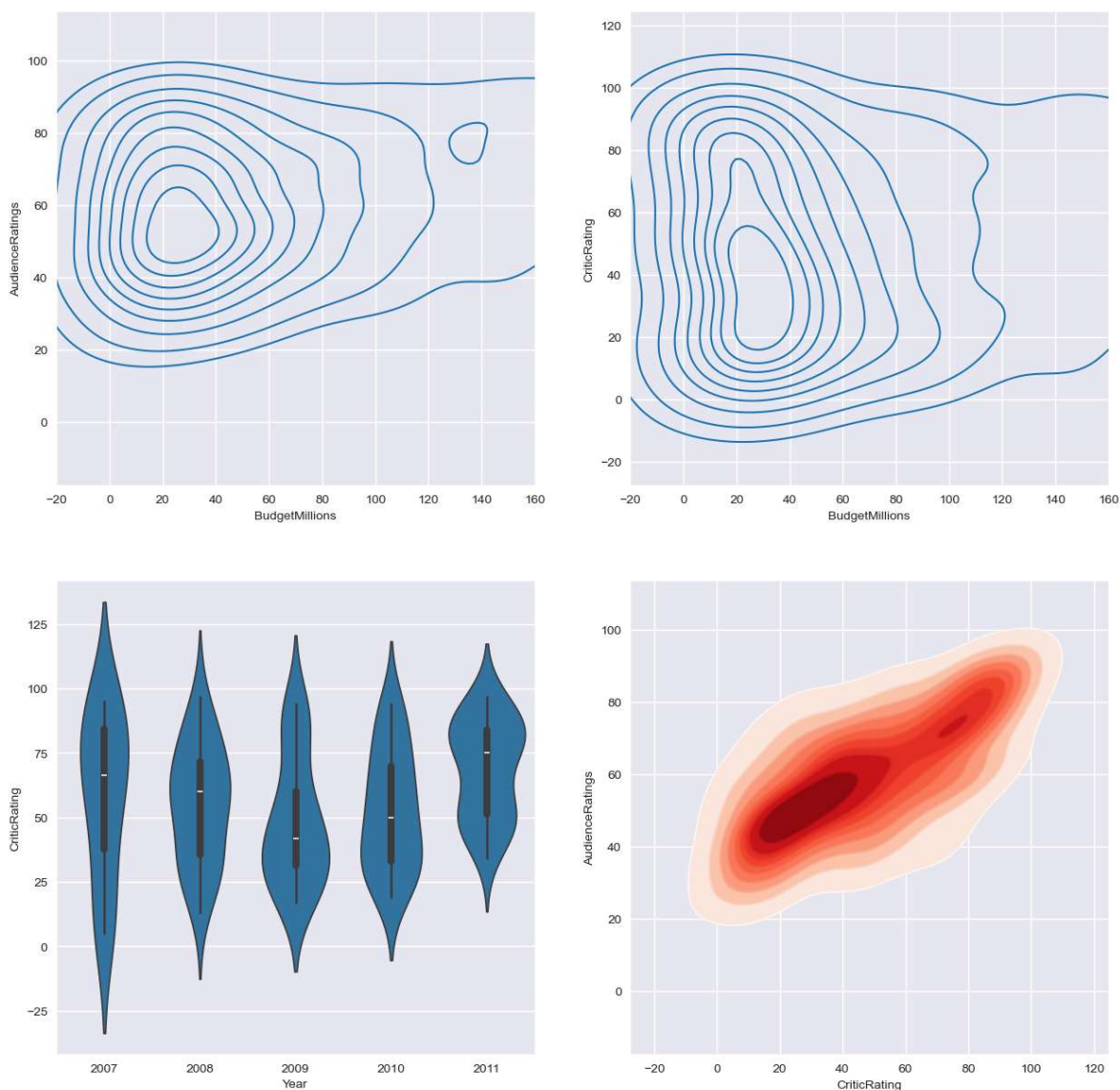
k1 = sns.kdeplot(x=mve.BudgetMillions,y=mve.AudienceRatings,ax=axes[0,0])
k2 = sns.kdeplot(x=mve.BudgetMillions,y=mve.CriticRating,ax = axes[0,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))
```

```

z = sns.violinplot(data=mve[mve.Genre=='Drama'], x='Year', y = 'CriticRating', a
k4 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings,shade = True,shade_low
k4b = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings,cmap='Reds',ax = axes
plt.show()

```



In [84]: *# How can you style your dashboard using different color map*

```

# python is not vectorize programming language
# Building dashboards (dashboard - combination of chats)

sns.set_style('dark',{'axes.facecolor':'black'})
f, axes = plt.subplots(2,2, figsize = (15,15))

#plot [0,0]
k1 = sns.kdeplot(x=mve.BudgetMillions,y=mve.AudienceRatings, \
                 shade = True, shade_lowest=True,cmap = 'inferno', \
                 ax = axes[0,0])
k1b = sns.kdeplot(x=mve.BudgetMillions, y=mve.AudienceRatings, \
                 cmap = 'cool',ax = axes[0,0])

#plot [0,1]
k2 = sns.kdeplot(x=mve.BudgetMillions,y=mve.CriticRating,\

```

```

        shade=True, shade_lowest=True, cmap='inferno',\
        ax = axes[0,1])
k2b = sns.kdeplot(x=mve.BudgetMillions,y=mve.CriticRating,\
        cmap = 'cool', ax = axes[0,1])

#plot[1,0]
z = sns.violinplot(data=mve[mve.Genre=='Drama'], \
        x='Year', y = 'CriticRating', ax=axes[1,0])

#plot[1,1]
k4 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings, \
        shade = True,shade_lowest=False,cmap='Blues_r', \
        ax=axes[1,1])

k4b = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRating, \
        cmap='gist_gray_r',ax = axes[1,1])

k1.set(xlim=(-20,160))
k2.set(xlim=(-20,160))

plt.show()

```

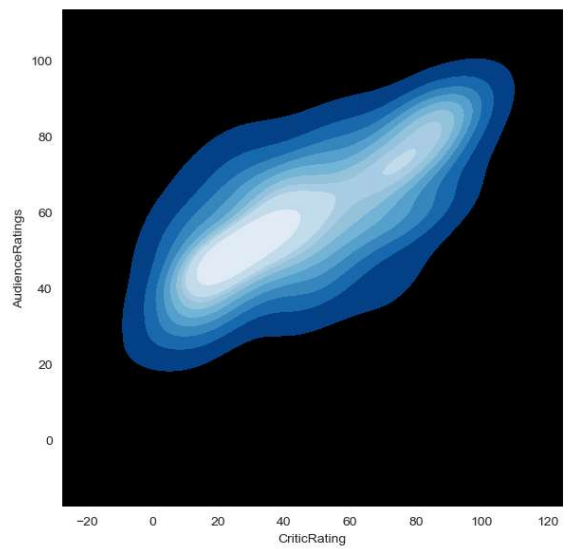
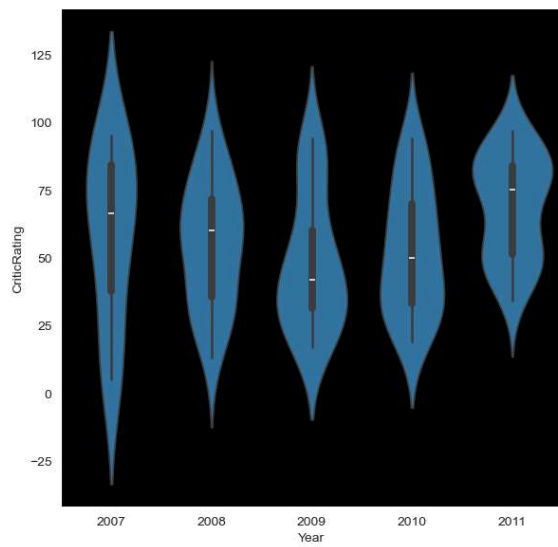
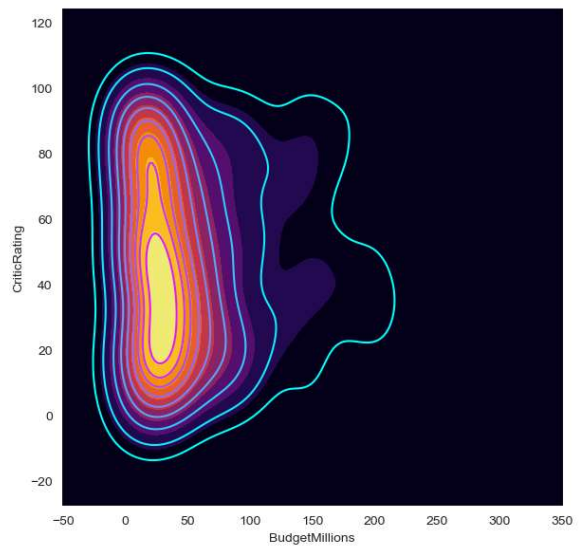
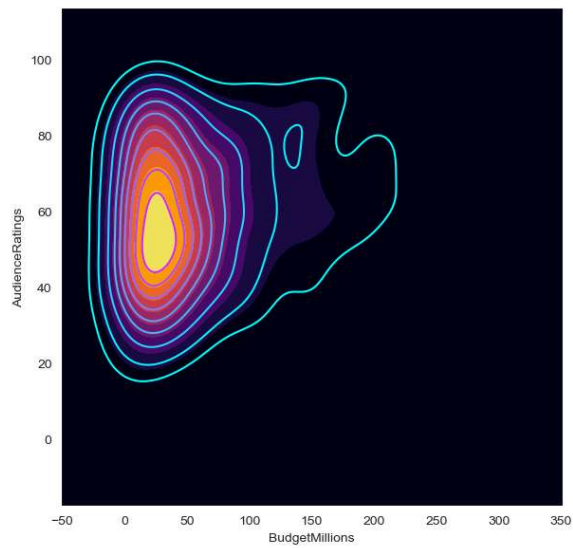
```

-----
AttributeError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_24464\4051722819.py in ?()
    28 k4 = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRatings, \
    29                 shade = True,shade_lowest=False,cmap='Blues_r', \
    30                 ax=axes[1,1])
    31
--> 32 k4b = sns.kdeplot(x=mve.CriticRating,y=mve.AudienceRating, \
    33                 cmap='gist_gray_r',ax = axes[1,1])
    34
    35

~\anaconda3\Lib\site-packages\pandas\core\generic.py in ?(self, name)
    6295         and name not in self._accessors
    6296         and self._info_axis._can_hold_identifiers_and_holds_name(name)
    6297     ):
    6298         return self[name]
-> 6299     return object.__getattribute__(self, name)

AttributeError: 'DataFrame' object has no attribute 'AudienceRating'

```



In [ ]: